

CURSO COMPLETO

PYTHON

IMPRESSONADOR

PARTE 6 • APROFUNDAMENTO, ESPECIALIZAÇÃO E EXTRAS

SUMÁRIO

MÓDULO 52 OUTRAS FERRAMENTAS AUXILIARES

6 2 7	Pytest – Framework de Testes em Python	4
-------	--	---

MÓDULO 54 INTRODUÇÃO A RECONHECIMENTO DE IMAGEM E FALA

6 3 0	Reconhecimento de Fala – SpeechRecognition	6 1
6 3 1	Reconhecimento de Imagem – OpenCV	7 1
6 3 2	Ciência de Dados – Reconhecimento de Imagem e Como quebrar CAPTCHA	8 1

MÓDULO 53 AULAS AUXILIARES DE FRONT-END

6 2 8	Introdução ao HTML	2 5
6 2 9	PyScript – Python no navegador – Python no HTML	4 7

MÓDULO 55 INTEGRAÇÃO PYTHON COM ARCGIS

6 3 3	Configurações para Integração Python com ArcGIS	8 8
6 1 4	Criando seus primeiros mapas com a API do ArcGIS	9 0
6 3 5	Integrando com Usuário e Senha no ArcGIS	9 2



PYTHON

Módulo 52

OUTRAS FERRAMENTAS AUXILIARES



O **Pytest** é uma estrutura de teste, um **framework** (conjunto de ferramentas) poderoso e popular para Python. Esse framework é utilizado principalmente para **testar códigos em Python**.

Ele oferece uma série de funcionalidades que facilitam a criação de testes completos, robustos e reutilizáveis.

O **Pytest** cria uma **lógica de testes** para o seu projeto, independentemente do tamanho dele, permitindo a **identificação de erros** e problemas de forma eficiente e automática.



Ferramentas e IDEs:

Para usar o Pytest, é recomendado que você utilize editores de código como o **PyCharm** ou o **VS Code**. Ao longo desse módulo, eu utilizarei o **PyCharm**, então, se você quiser usar a mesma IDE que eu estarei utilizando, opte por ele.

Ambiente Virtual:

Além disso, é recomendado fazer todo esse procedimento dentro de um **ambiente virtual**. Isso ajuda a otimizar os testes do Pytest.

Instalação:

O primeiro passo será instalar o Pytest. Você pode fazer isso executando o comando **pip install pytest** no **terminal** do **PyCharm** ou de qualquer outro editor de código.

```
PS C:\Users\diego\PycharmProjects\PyTest> pip install pytest
```

Funcionamento do Pytest:

Feita a instalação, o Pytest estará pronto para executar testes em seu projeto. Para executar todos os testes, basta usar o comando **pytest** no terminal do seu editor de código.

```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 0 items

===== no tests ran in 0.01s =====
```

O Pytest percorrerá e identificará automaticamente os **arquivos configurados para teste** dentro do projeto e realizará os testes necessários.

Como acabamos de criar o projeto e ainda não há arquivos de teste, ele apresentará a mensagem **Collected 0 items**, indicando que **nenhum item de teste** foi encontrado.

Ao final do processo, ele também exibirá o tempo gasto com os testes executados. No nosso exemplo, como não houve nenhum teste, o tempo foi de **0.01 segundos**.

Criando o Projeto – Sistema de Gerenciamento:

Como exemplo para esta aula, criaremos algumas **funções** que fariam parte de um **sistema de gerenciamento** de uma empresa.

Dentro desse sistema, precisaremos de uma **função para calcular o faturamento**, outra para calcular o **custo**, e uma para calcular o **lucro**.

Começaremos definindo a função que irá **calcular o lucro**. Esta função receberá como parâmetros o **faturamento** e o **custo**, e retornará como resposta o **lucro**, que nada mais é do que o faturamento menos o custo.

```
1  #Sistema Gerencial
2
3  #Calcular o Faturamento
4
5  #Calcular o Custo
6
7  #Calcular o Lucro
8  def calcular_lucro(faturamento, custo):
9      lucro = faturamento - custo
10     return lucro
```

Configurando Arquivos para Teste:

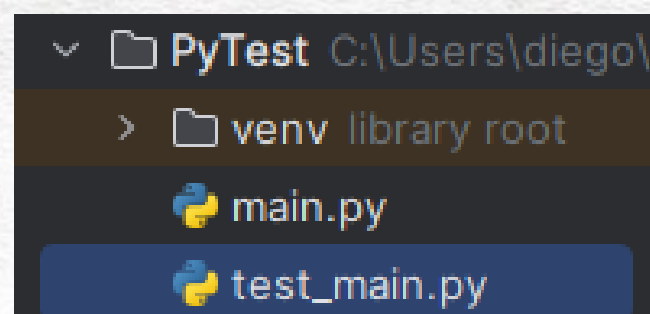
Agora que já temos a nossa primeira função criada, podemos criar nosso arquivo de teste.

Como já foi mencionado, o **Pytest** identifica automaticamente os arquivos configurados para teste dentro do projeto.

Esses arquivos precisam seguir uma convenção de nomenclatura que fará com que o Pytest os identifique corretamente: eles precisam começar com **test_** ou terminar com **_test**.

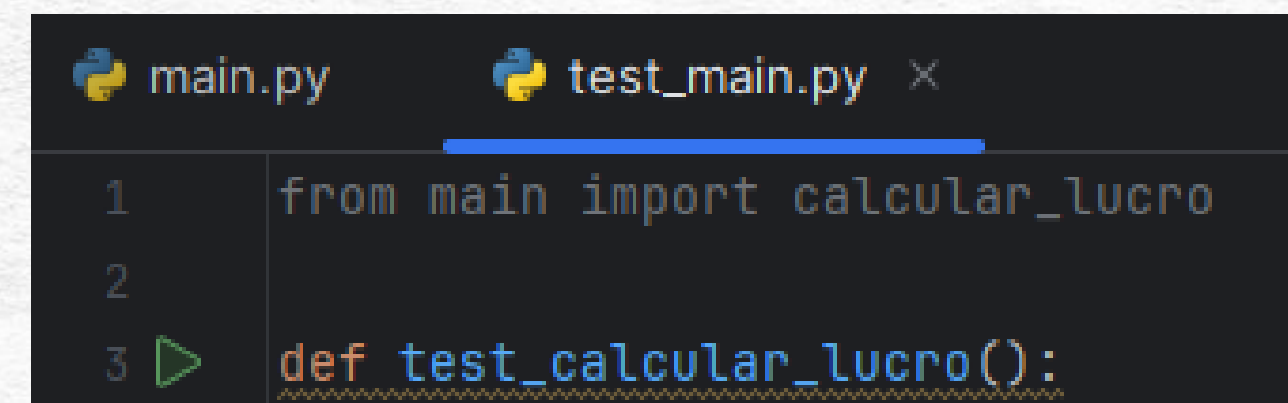
Ou seja, o Pytest identificará os arquivos para teste caso o nome esteja definido de uma dessas duas formas: **test_nome_arquivo.py** ou **nome_arquivo_test.py**

Então, para criar o arquivo de teste do arquivo principal onde estão as funções, vamos criar o arquivo **test_main.py**.



Você poderia criar o arquivo com o nome que quisesse desde que ele começasse com **test_** ou terminasse com **_test**, mas é uma boa prática utilizar um nome que reflita o nome do **arquivo original** para facilitar a manutenção e a leitura do código.

Dentro do arquivo de testes, vamos importar a função **calcular_lucro** a partir do arquivo **main** e criaremos a função de teste. Essa função também deve respeitar a nomenclatura adequada para o **Pytest**, então receberá o nome de **test_calcular_lucro**.

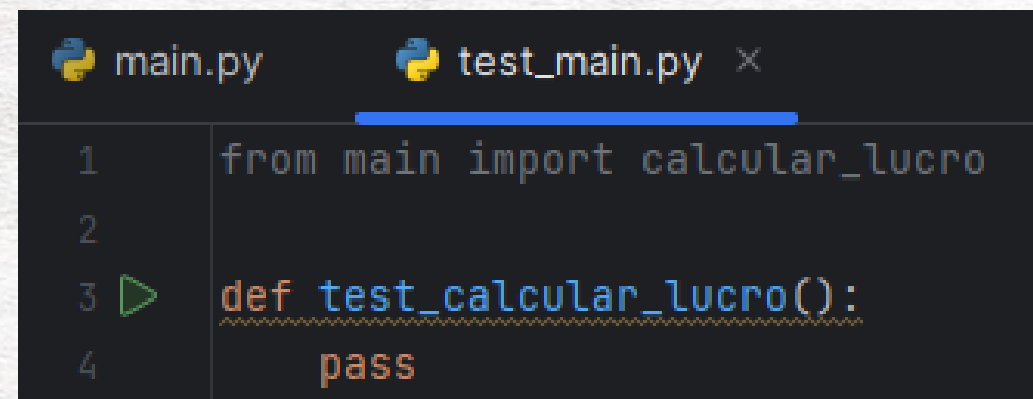


Dentro dessa função definiremos o tipo de teste que desejamos realizar.

Testando o Arquivo:

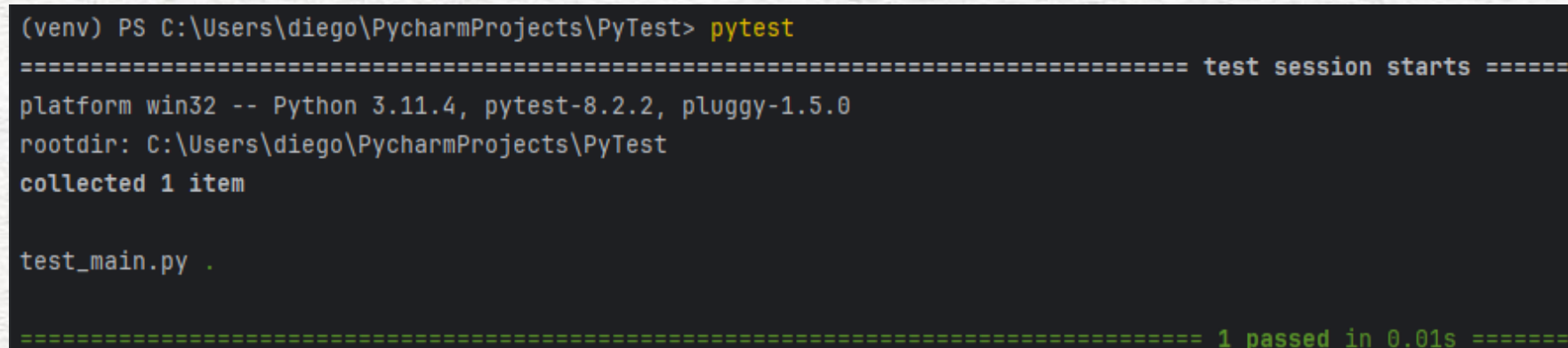
Vamos começar com um teste simples que passará sem erros. Depois, adicionaremos um teste que propositalmente causará um erro para entender como o Pytest reporta falhas.

Para começar, adicionaremos apenas uma instrução **pass**, para que o código execute sem erros.



```
main.py test_main.py x
1 from main import calcular_lucro
2
3 def test_calcular_lucro():
4     pass
```

Feito isso, execute o comando `pytest` no terminal do seu editor de códigos.



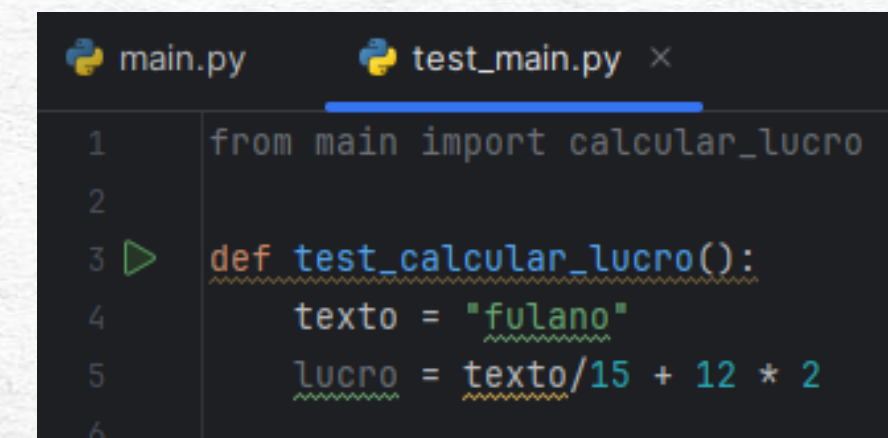
```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 1 item

test_main.py .

===== 1 passed in 0.01s =====
```

Observe que agora ele já identificou o arquivo e executou o teste necessário. Como a execução foi apenas uma instrução `pass`, o código funcionou como esperado, então o Pytest retornou a resposta de 1 passed ao final, indicando que o código passou no teste.

Vamos alterar essa função para um código que indicará um erro.



```
main.py test_main.py x
1 from main import calcular_lucro
2
3 def test_calcular_lucro():
4     texto = "fulano"
5     lucro = texto/15 + 12 * 2
6
```

Claramente essa função retornará um erro pois estamos tentando dividir uma **string** (um texto) por um **número inteiro**. Mas vamos executar o **Pytest** para visualizar como ele indica esses erros.

Testando o Arquivo:

Execute novamente o comando **pytest** no terminal do seu editor de códigos.

```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 1 item

test_main.py F

===== FAILURES =====
test_calcular_lucro

def test_calcular_lucro():
    texto = "fulano"
> lucro = texto/15 + 12 * 2
E      TypeError: unsupported operand type(s) for /: 'str' and 'int'

test_main.py:5: TypeError
===== short test summary info =====
FAILED test_main.py::test_calcular_lucro - TypeError: unsupported operand type(s) for /: 'str' and 'int'
===== 1 failed in 0.05s =====
```

Vamos entender mais a fundo essa resposta do Pytest.

Identificação do Erro: O Pytest informa em qual arquivo foi encontrado o erro, indicando o nome do arquivo seguido da letra **F**.

Detalhes do Erro: Logo abaixo, o Pytest apresenta em qual função ocorreu a falha e indica a linha de código em que houve o erro, com a explicação do tipo de erro. Nesse caso, um **TypeError** pois estamos tentando dividir uma string por um número inteiro.

Resumo do Teste: Por fim, ele apresenta um resumo das informações do teste. Então temos novamente ele informando que dentro do **test_main.py** houve um erro na função **test_calcular_lucro** do tipo **TypeError**.

Repare que agora ele indica **1 failed**, mostrando que um dos códigos falhou em sua execução correta.

Testando o Arquivo:

Se observarmos o teste em que deu tudo certo, veremos que ao lado do nome do arquivo, o Pytest apenas adiciona um **ponto** (.), indicando que aquele arquivo passou nos testes.

```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 1 item

test_main.py .

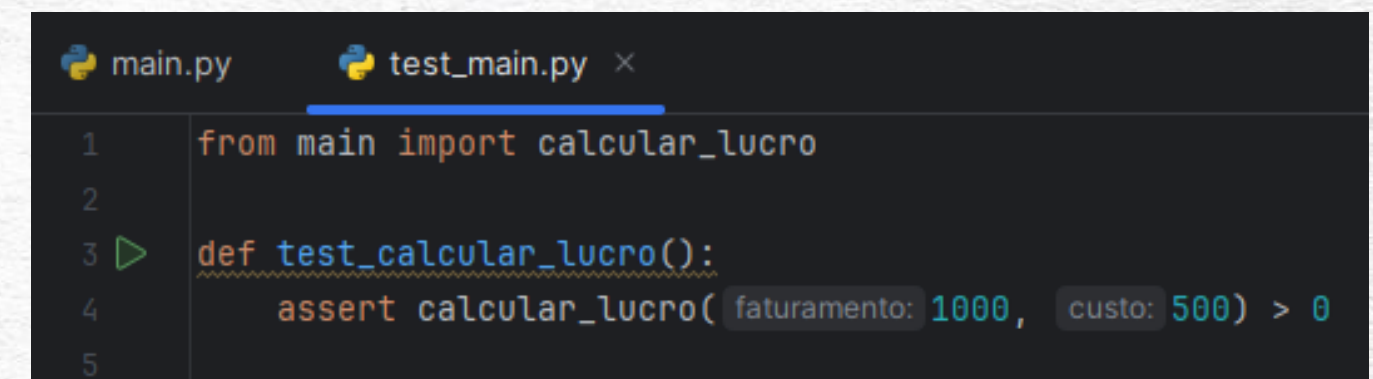
===== 1 passed in 0.01s =====
```

Agora que já compreendemos o funcionamento do Pytest, vamos ver como aplicá-lo corretamente para testar se a função **calcular_lucro** está funcionando como o esperado.

Para fazer isso, utilizaremos um **assert**, uma declaração que verifica se uma condição é **verdadeira** ou **falsa**. Se a condição for falsa, o **assert** dispara uma exceção (**AssertionError**).

O **assert** é muito útil pois nos permite verificar automaticamente se a saída de uma função está correta.

Por exemplo, podemos criar um teste para verificar se ao executar a função **calcular_lucro** com os valores **1000** e **500** o resultado será **maior do que 0**.



```
main.py  test_main.py x
1  from main import calcular_lucro
2
3  def test_calcular_lucro():
4      assert calcular_lucro(faturamento: 1000, custo: 500) > 0
5
```

Executando o Pytest teremos a mensagem de **sucesso** no teste.

```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 1 item

test_main.py .

===== 1 passed in 0.01s =====
```

Essa é uma forma de verificar se a lógica e a construção da sua função estão funcionando corretamente.

Testando o Arquivo:

Para que você possa compreender melhor as aplicações do **Pytest** e o uso do **assert**, definiremos agora a função responsável por calcular o faturamento dentro do arquivo **main.py**.

```
1  #Sistema Gerencial
2
3  #Calcular o Faturamento
4  def calcular_faturamento():
5      vendas = [10, 20, 30, 40, 50]
6      faturamento = sum(vendas)
7      return f"O faturamento foi de {faturamento}"
```

Essa função recebe uma **lista de vendas**, calcula o faturamento da empresa através da função **sum** e retorna uma mensagem informando o faturamento.

Podemos testar seu funcionamento executando um **print** dela.

```
16 print(calcular_faturamento())
```

```
C:\Users\diego\PycharmProjects
O faturamento foi de 150
```

Veja que a função foi executada normalmente, sem nenhum erro, e nos apresentou a mensagem **"O faturamento foi de 150"** como esperado para os valores presentes na lista vendas.

A princípio, quando você observa seu código, parece que ele está correto, pois tanto a função **calcular_faturamento** quanto a função **calcular_lucro** estão funcionando corretamente.

```
main.py x test_main.py
1  #Sistema Gerencial
2
3  #Calcular o Faturamento
4  def calcular_faturamento():
5      vendas = [10, 20, 30, 40, 50]
6      faturamento = sum(vendas)
7      return f"O faturamento foi de {faturamento}"
8
9  #Calcular o Custo
10
11 #Calcular o Lucro
12 def calcular_lucro(faturamento, custo):
13     lucro = faturamento - custo
14     return lucro
```

Testando o Arquivo:

No entanto, as duas funções estão funcionando de forma correta **separadamente**. Porém, em algum momento dentro do sistema de gerenciamento, elas precisarão **interagir**, pois a função **calcular_lucro** usa do **faturamento** para que ela possa funcionar.

Para verificar se a interação entre as funções está correta, podemos utilizar o Pytest.

Dentro do arquivo **test_main.py**, vamos importar a função **calcular_faturamento** e, dentro da função **test_calcular_lucro**, definiremos a lógica de interação entre as duas funções.

Ou seja, o faturamento será obtido a partir da função **calcular_faturamento** e o resultado dessa função será usado dentro da função **calcular_lucro**.

```
main.py  test_main.py x
1  from main import calcular_lucro, calcular_faturamento
2
3  def test_calcular_lucro():
4      faturamento = calcular_faturamento()
5      assert calcular_lucro(faturamento, custo=500) > 0
```

Com a lógica criada, podemos executar o **Pytest** e, com isso, obteremos uma mensagem de erro indicando que há uma **falha** no arquivo **test_main.py**, dentro da função **test_calcular_lucro**, na linha em que definimos o **assert** com a chamada da função **calcular_lucro**.

```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 1 item

test_main.py F

===== FAILURES =====
----- test_calcular_lucro -----

def test_calcular_lucro():
    faturamento = calcular_faturamento()
>    assert calcular_lucro(faturamento, 500) > 0
test_main.py:5:

faturamento = '0 faturamento foi de 150', custo = 500

def calcular_lucro(faturamento, custo):
>    lucro = faturamento - custo
E       TypeError: unsupported operand type(s) for -: 'str' and 'int'

main.py:13: TypeError
===== short test summary info =====
FAILED test_main.py::test_calcular_lucro - TypeError: unsupported operand type(s) for -: 'str' and 'int'
===== 1 failed in 0.04s =====
```

Testando o Arquivo:

```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 1 item

test_main.py F

===== FAILURES =====
test_calcular_lucro

def test_calcular_lucro():
    faturamento = calcular_faturamento()
    > assert calcular_lucro(faturamento, 500) > 0

test_main.py:5:
-----
faturamento = 'O faturamento foi de 150', custo = 500

def calcular_lucro(faturamento, custo):
    > lucro = faturamento - custo
E       TypeError: unsupported operand type(s) for -: 'str' and 'int'

main.py:13: TypeError
===== short test summary info =====
FAILED test_main.py::test_calcular_lucro - TypeError: unsupported operand type(s) for -: 'str' and 'int'
===== 1 failed in 0.04s =====
```

Dentro dessa explicação, temos os valores atribuídos às variáveis dentro da função **calcular_lucro**, onde faturamento é a string **"O faturamento foi de 150"** e custo é o número **500**.

Em seguida, temos a indicação do tipo de erro, que foi um **TypeError**, ao chamar a função **calcular_lucro**. Isso ocorreu porque tentamos subtrair um número inteiro de uma string, e essa não é uma operação possível.

Além disso, logo abaixo temos a explicação do erro.

Testando o Arquivo:

Com essa informação, podemos ajustar a função `calcular_faturamento` para que, ao invés de retornar uma mensagem com o valor do faturamento, ela retorne apenas o **valor** em si.

```
main.py x test_main.py
1  #Sistema Gerencial
2  ~~~~~
3  #Calcular o Faturamento
4  ~~~~~
5  def calcular_faturamento():
6      vendas = [10, 20, 30, 40, 50]
7      faturamento = sum(vendas)
8      return faturamento
```

Isso soluciona o erro indicado pelo Pytest. Então, podemos executá-lo novamente para verificar se agora as duas funções estão interagindo corretamente.

```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 1 item

test_main.py F

===== FAILURES =====
----- test_calcular_lucro -----

    def test_calcular_lucro():
        faturamento = calcular_faturamento()
        > assert calcular_lucro(faturamento, 500) > 0
E       assert -350 > 0
E       + where -350 = calcular_lucro(150, 500)

test_main.py:5: AssertionError
===== short test summary info =====
FAILED test_main.py::test_calcular_lucro - assert -350 > 0
===== 1 failed in 0.04s =====
```

Repare que, mesmo realizando a mudança na função `calcular_faturamento`, obtivemos uma **nova indicação de erro** dentro do Pytest. Isso porque definimos que o teste deveria verificar se o valor retornado pela função `calcular_lucro` é **maior do que 0**.

O que não acontece, e por isso ele indica que o valor atual retornado pela função é de **-350**, que não é maior do que 0.

Testando o Arquivo:

Se dentro do seu programa for permitido que a empresa tenha um **lucro negativo**, ou seja, um **prejuízo** dentro do período, e esse valor será aceito pela função, você pode trocar a verificação feita pelo **assert** para saber se o tipo retornado pela função é igual a um **número inteiro**, ou um **número float**, por exemplo.

```
main.py  test_main.py ×
1  from main import calcular_lucro, calcular_faturamento
2
3  def test_calcular_lucro():
4      faturamento = calcular_faturamento()
5      assert type(calcular_lucro(faturamento, custo: 500)) == int
6
```

Executando o Pytest agora, o resultado será um sucesso.

```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 1 item

test_main.py .

===== 1 passed in 0.01s =====
```

Para continuar o nosso exemplo, vamos considerar que dentro desse sistema não serão considerados os casos em que a empresa teve prejuízo. Então, vamos retornar o teste como estava.

```
main.py  test_main.py ×
1  from main import calcular_lucro, calcular_faturamento
2
3  def test_calcular_lucro():
4      faturamento = calcular_faturamento()
5      assert calcular_lucro(faturamento, custo: 500) > 0
6
```

Benefícios do Pytest em Projetos Maiores:

Quanto maior for o projeto em que você está trabalhando, com mais arquivos e interações entre eles e as funções presentes, mais vantajoso será o uso do Pytest.

Desde que as lógicas de teste sejam definidas de forma clara dentro dos arquivos de teste, você será capaz de identificar o tipo de erro, onde e por que ele ocorreu.

Até o momento, só definimos uma lógica de teste dentro do nosso arquivo de teste, mas poderíamos criar **mais de um teste**.

Primeiro, vamos aumentar os valores da lista de vendas no nosso arquivo **main.py** para passarmos pelo primeiro teste da função **test_calcular_lucro**.

```
main.py x test_main.py
1 #Sistema Gerencial
2
3 #Calcular o Faturamento
4 def calcular_faturamento():
5     vendas = [1000, 2000, 3000, 4000, 5000]
6     faturamento = sum(vendas)
7     return faturamento
```

Agora, de volta ao arquivo **test_main.py**, vamos definir um segundo teste que verificará apenas se a função **calcular_faturamento** está retornando um valor do tipo inteiro.

```
main.py x test_main.py x
1 from main import calcular_lucro, calcular_faturamento
2
3 def test_calcular_lucro():
4     faturamento = calcular_faturamento()
5     assert calcular_lucro(faturamento, custo=500) > 0
6
7 def test_calcular_faturamento():
8     assert type(calcular_faturamento()) == int
```

Executando o Pytest agora, ele coletará dois itens para teste e indicará se eles passaram ou não.

```
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 2 items

test_main.py ..

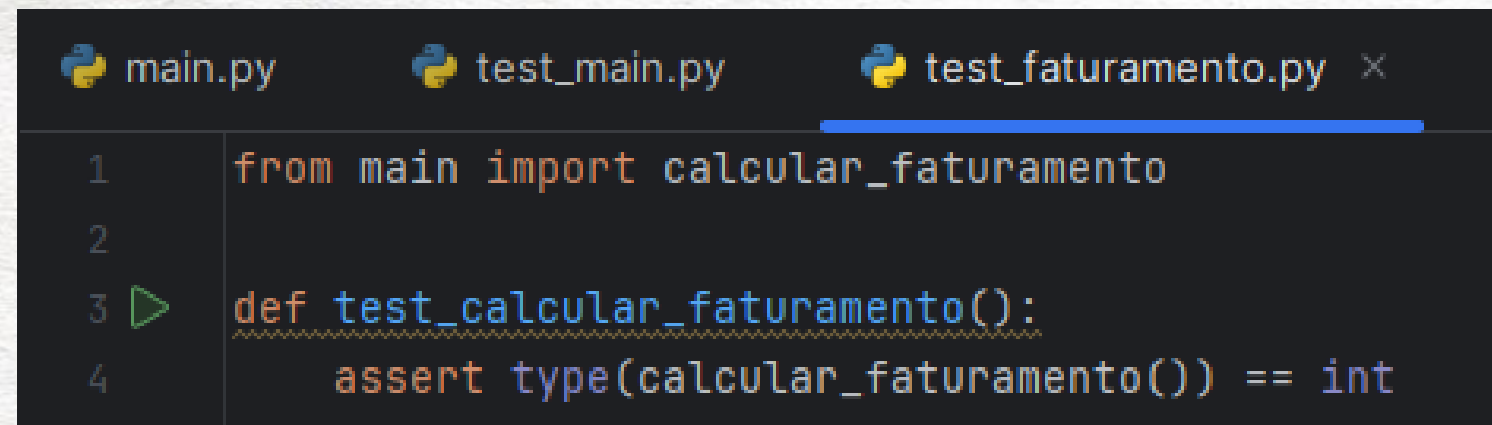
===== 2 passed in 0.01s =====
```

Repare que, como os dois testes obtiveram sucesso, ele indica **dois pontos** ao lado do nome do arquivo **test_main.py**.

Dividindo Testes em Múltiplos Arquivos:

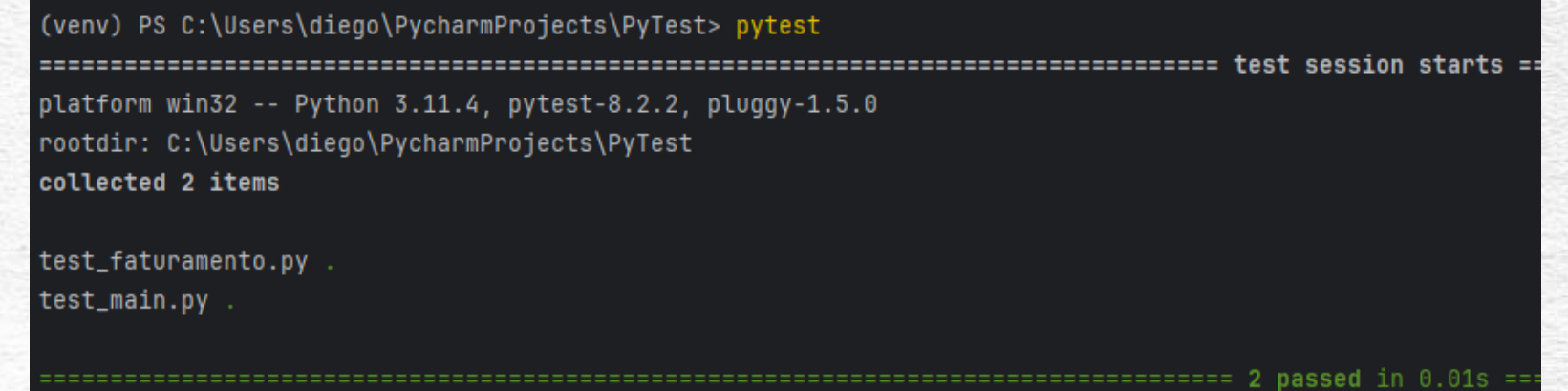
Em projetos grandes, você não vai querer ter todos os testes definidos em um único arquivo. É possível dividir os testes em **múltiplos arquivos**, o que é especialmente útil, pois permite que você selecione quais arquivos deseja testar.

Vamos criar um segundo arquivo chamado **test_faturamento.py** e colocar dentro dele a função **test_calcular_faturamento**.



```
1 from main import calcular_faturamento
2
3 def test_calcular_faturamento():
4     assert type(calcular_faturamento()) == int
```

Se apenas executarmos o Pytest, ele realizará os testes tanto no arquivo **test_main.py** quanto no **test_faturamento.py**.

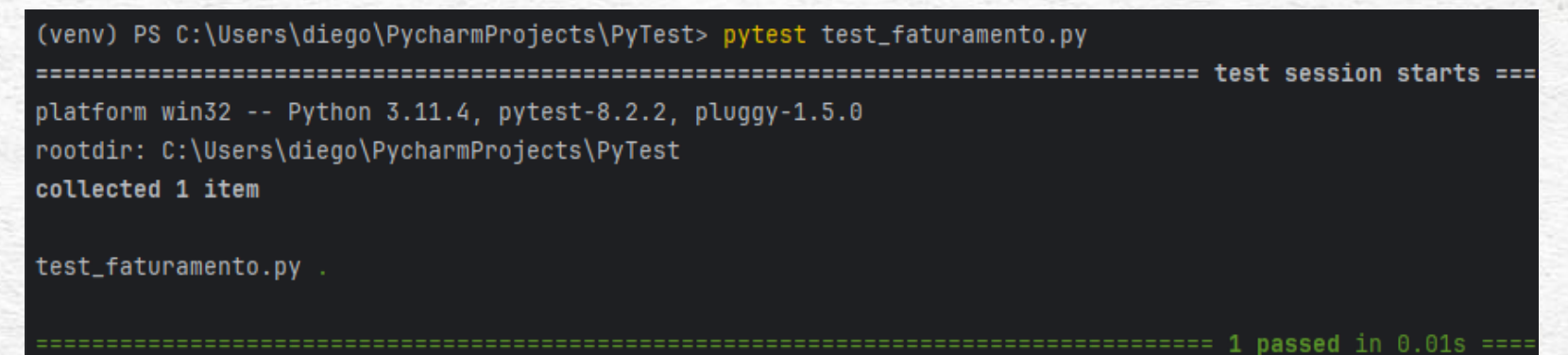


```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 2 items

test_faturamento.py .
test_main.py .

===== 2 passed in 0.01s =====
```

Porém, em alguns casos, você só desejará ou precisará testar partes específicas do seu projeto. Para esses casos, você pode passar como parâmetro para o Pytest o **nome do arquivo** onde está a lógica/funcionamento que você quer verificar.



```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest test_faturamento.py
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
collected 1 item

test_faturamento.py .

===== 1 passed in 0.01s =====
```

Marcadores do Pytest:

Uma funcionalidade interessante do Pytest são os **marcadores**. Eles permitem categorizar e organizar os testes de forma flexível.

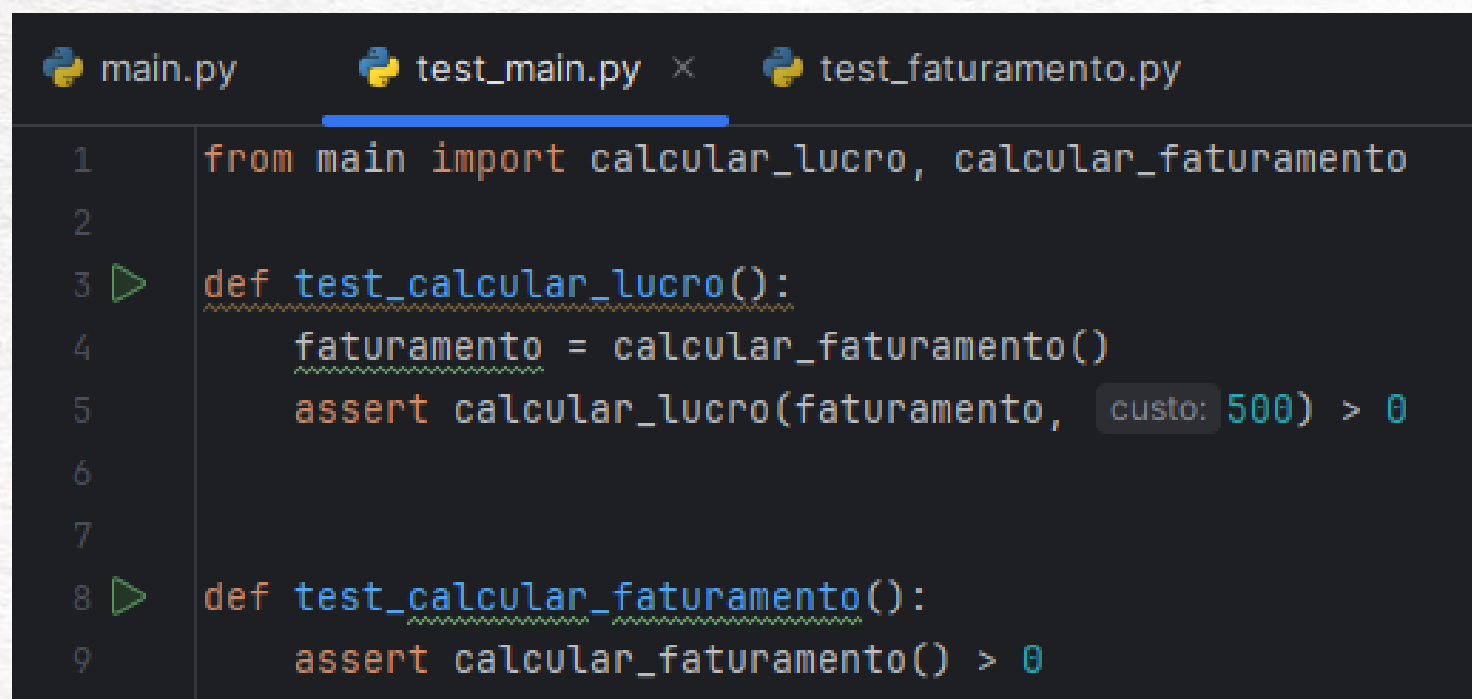
Os marcadores servem para agrupar testes a partir de um **critério em comum** (que você define). Assim como podemos executar o Pytest apenas para um determinado arquivo, podemos também executá-lo para um determinado marcador.

Vamos começar criando uma nova função para o faturamento dentro do arquivo **test_main.py**.

Essa função receberá um marcador juntamente com a função **test_calcular_faturamento** presente no arquivo **test_faturamento.py**. Dessa forma estaremos categorizando apenas os testes envolvendo a função **calcular_faturamento**.

Para utilizar essa funcionalidade do Pytest, primeiro precisamos criar um arquivo chamado **pytest.ini**. Dentro desse arquivo, utilizaremos o seguinte código:

```
[pytest]
addopts = --strict-markers
markers =
```



```
main.py  test_main.py  test_faturamento.py
1  from main import calcular_lucro, calcular_faturamento
2
3  def test_calcular_lucro():
4      faturamento = calcular_faturamento()
5      assert calcular_lucro(faturamento, custo=500) > 0
6
7
8  def test_calcular_faturamento():
9      assert calcular_faturamento() > 0
```



Marcadores do Pytest:

Esse é um código padrão para utilizar os marcadores com o Pytest. Dentro dele, definimos na opção **markers** os nomes das categorias que iremos agrupar os nossos testes. Nesse caso, vamos criar o marcador **faturamento**.

```
main.py test_main.py pytest.ini x
1 [pytest]
2 addopts = --strict-markers
3 markers =
4     faturamento
```

Com nosso marcador criado, podemos importar o **Pytest** dentro do arquivo **test_main.py** e, antes da declaração da função **test_calcular_faturamento**, definiremos um decorador da seguinte forma: **@pytest.mark.faturamento**

```
main.py test_main.py x test_faturamento.py
1 from main import calcular_lucro, calcular_faturamento
2 import pytest
3
4 def test_calcular_lucro():
5     faturamento = calcular_faturamento()
6     assert calcular_lucro(faturamento, custo=500) > 0
7
8 @pytest.mark.faturamento
9 def test_calcular_faturamento():
10    assert calcular_faturamento() > 0
```

Faremos o mesmo no arquivo **test_faturamento.py**.

```
main.py test_main.py test_faturamento.py x
1 from main import calcular_faturamento
2 import pytest
3
4 @pytest.mark.faturamento
5 def test_calcular_faturamento():
6     assert type(calcular_faturamento()) == int
```

Marcadores do Pytest:

Agora podemos executar o Pytest passando como argumento para ele o marcador faturamento. Para indicar que estamos passando um marcador, utilizamos a opção **-m**.

```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest -m faturamento
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
configfile: pytest.ini
collected 3 items / 1 deselected / 2 selected

test_faturamento.py .
test_main.py .

===== 2 passed, 1 deselected in 0.01s =====
```

Repare que ele mostra que foram **coletados três itens**, pois temos três funções definidas nos arquivos de testes, mas uma delas não foi selecionada (**deselected**), que é a função que calcula o lucro.

Das duas funções selecionadas, que possuem o marcador faturamento, ambas passaram no teste.

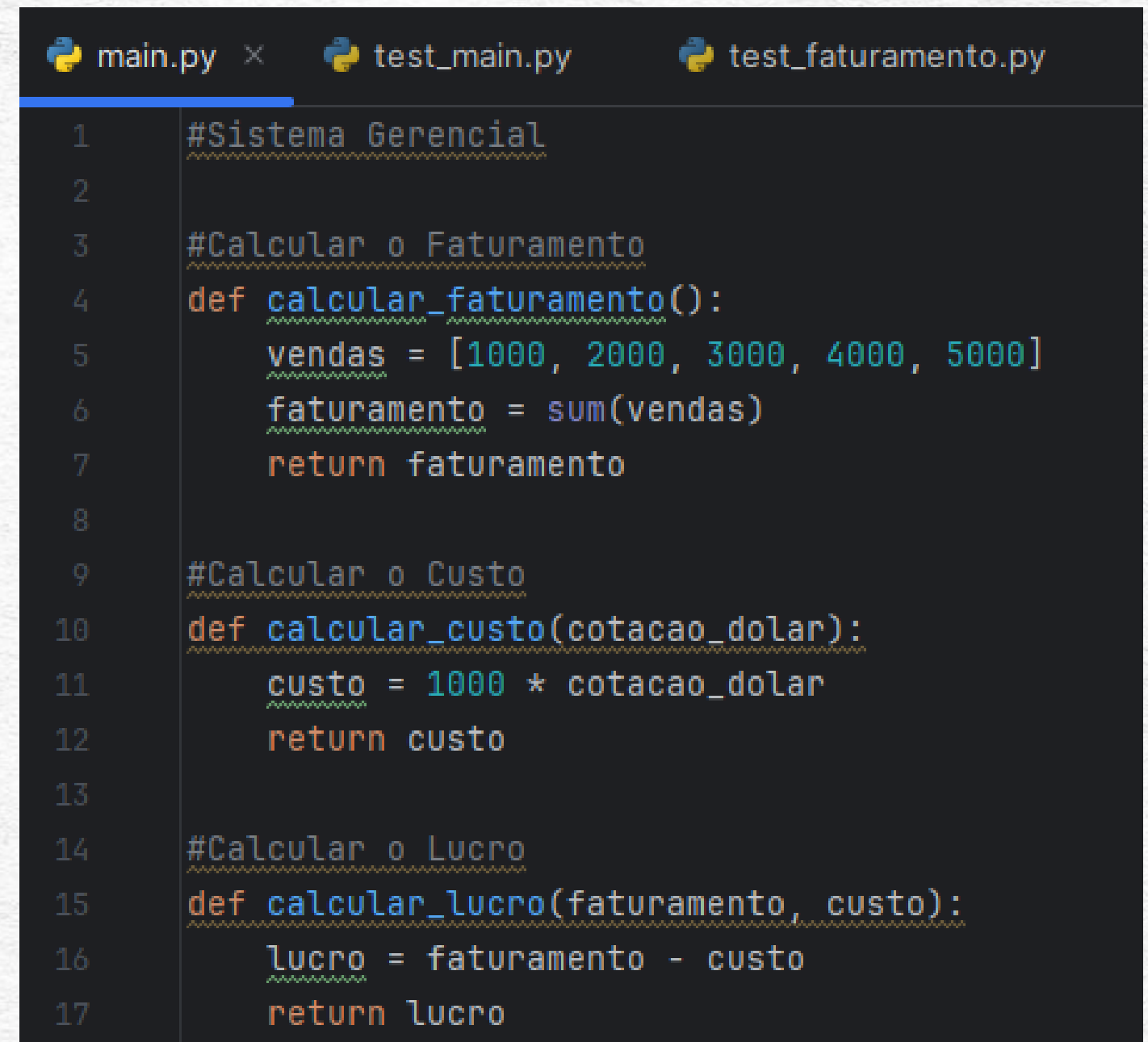
Fixtures no Pytest:

O último conceito referente ao Pytest que eu quero te mostrar são as **fixtures**. As **fixtures** são uma forma de preparar o ambiente de teste ou fornecer os dados e condições necessárias para que o teste seja executado.

O Pytest utiliza as **fixtures** para gerenciar o que precisa ser executado antes dos testes. Assim como usamos o **decorator** `@pytest.mark.nome_do_marcador` para definir um marcador, também usaremos um **decorator**, `@pytest.fixture`, para definir uma fixture.

Vamos entender como isso funciona na prática. Dentro do arquivo `main.py`, vamos definir a função responsável por calcular o custo da empresa.

Para o nosso exemplo, vamos considerar que essa empresa tem um **custo fixo de 1000 dólares**, porém o faturamento obtido por ela é dado em reais. Dessa forma, para calcular o custo da empresa em reais, precisamos pegar o **valor fixo de 1000** e **multiplicar pela cotação atual do dólar**, que a função receberá como argumento.



```
1  #Sistema Gerencial
2
3  #Calcular o Faturamento
4  def calcular_faturamento():
5      vendas = [1000, 2000, 3000, 4000, 5000]
6      faturamento = sum(vendas)
7      return faturamento
8
9  #Calcular o Custo
10 def calcular_custo(cotacao_dolar):
11     custo = 1000 * cotacao_dolar
12     return custo
13
14 #Calcular o Lucro
15 def calcular_lucro(faturamento, custo):
16     lucro = faturamento - custo
17     return lucro
```

Fixtures no Pytest:

Sendo assim, não faz sentido que a empresa tenha um **custo negativo**, então podemos criar um teste dentro do arquivo **test_main.py** que verificará se a função **calcular_custo** retornará um valor maior do que 0.

Porém, para que esse teste funcione, ele também precisa receber como parâmetro a **cotação do dólar**.

O que acontece nesse caso é que, geralmente, a cotação do dólar será obtida através de uma **API externa** ao seu sistema, ou seja, você precisará puxar essa informação de uma API com a biblioteca **Requests** do Python.

Então, antes do teste responsável por verificar se a função **calcular_custo** está funcionando conforme o esperado, precisamos definir uma função que calculará a cotação do dólar, e é essa função que será a nossa **fixture**.

```
main.py  test_main.py  test_faturamento.py
1  from main import calcular_lucro, calcular_faturamento, calcular_custo
2  import pytest
3
4  def test_calcular_lucro():
5      faturamento = calcular_faturamento()
6      assert calcular_lucro(faturamento, custo=500) > 0
7
8  @pytest.mark.faturamento
9  def test_calcular_faturamento():
10     assert calcular_faturamento() > 0
11
12  def test_calcular_custo(cotacao_dolar):
13     assert calcular_custo(cotacao_dolar) > 0
```

Fixtures no Pytest:

A **fixture** precisa receber o mesmo nome do parâmetro necessário para o teste que irá utilizá-la.

Em um sistema real e completo, dentro dessa fixture faríamos a requisição para a API para obter a cotação do dólar. Para deixar nosso exemplo mais direto, eu definirei o **return** dessa fixture como sendo o valor 5.

Executando o Pytest teremos que todos os testes passaram com sucesso.

```
main.py  test_main.py  test_faturamento.py
1  from main import calcular_lucro, calcular_faturamento, calcular_custo
2  import pytest
3
4  def test_calcular_lucro():
5      faturamento = calcular_faturamento()
6      assert calcular_lucro(faturamento, custo=500) > 0
7
8  @pytest.mark.faturamento
9  def test_calcular_faturamento():
10     assert calcular_faturamento() > 0
11
12  @pytest.fixture
13  def cotacao_dolar():
14     #Aqui você criaria a requisição para pegar a cotação do dólar da API
15     return 5
16
17  def test_calcular_custo(cotacao_dolar):
18     assert calcular_custo(cotacao_dolar) > 0
```

```
(venv) PS C:\Users\diego\PycharmProjects\PyTest> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
configfile: pytest.ini
collected 4 items

test_faturamento.py .
test_main.py ...

===== 4 passed in 0.01s =====
```

Fixtures no Pytest:

Se removêssemos a fixture, obteríamos uma falha informando que a **fixture cotacao_dolar** não foi encontrada. E ela é essencial para a execução correta do teste **test_calcular_custo**.

```
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-8.2.2, pluggy-1.5.0
rootdir: C:\Users\diego\PycharmProjects\PyTest
configfile: pytest.ini
collected 4 items

test_faturamento.py .
test_main.py ..E

===== ERRORS =====
----- ERROR at setup of test_calcular_custo -----
file C:\Users\diego\PycharmProjects\PyTest\test_main.py, line 14
    def test_calcular_custo(cotacao_dolar):
E       fixture 'cotacao_dolar' not found
>       available fixtures: cache, capfd, capfdbinary, caplog, capsys, capsysbinary, doctest_namespace, monkeypatch, pytest
tribune, recwarn, tmp_path, tmp_path_factory, tmpdir, tmpdir_factory
>       use 'pytest --fixtures [testpath]' for help on them.

C:\Users\diego\PycharmProjects\PyTest\test_main.py:14
===== short test summary info =====
ERROR test_main.py::test_calcular_custo
===== 3 passed, 1 error in 0.02s =====
```

Sempre que um teste que você fizer depender de uma informação que vem de fora dele, você poderá criar uma **fixture**. Isso será mais comum em casos em que precisamos consumir informações que estão vindo de outro sistema, API, banco de dados, etc.

As fixtures são interessantes pois permitem uma melhor organização, permitindo que você identifique se a falha está vindo a partir dela, ou do teste.

Por exemplo, se a API retornasse o texto **"5"** ao invés do número **5**, você conseguiria identificar esse erro dentro da sua fixture e tratá-lo corretamente antes de utilizar dentro do teste.

Nesse módulo você pode ver algumas das funcionalidades presentes dentro do Pytest. Mas é aconselhável ler e conhecer a [documentação](#) desse framework para que você possa se aprofundar ainda mais nas capacidades dele.



PYTHON

Módulo 53

AULAS AUXILIARES DE FRONT-END



Neste módulo, veremos o que é **HTML** e qual a importância dessa linguagem para você que está estudando Python.

O que é HTML?

HTML significa **Hypertext Markup Language**. É uma linguagem de marcação fundamental para a construção de sites. Junto com o **CSS**, ela atua na construção da parte visual das páginas na internet.

O HTML é responsável por **construir a estrutura básica do site**, formando o “**esqueleto**” dele por meio de **tags**. Essas **tags** permitem incluir em uma **página web** os títulos, parágrafos, imagens e demais elementos.

Todos os sites que você acessa na internet são construídos utilizando **HTML** e **CSS** que, junto com o **JavaScript**, são responsáveis pelo **desenvolvimento front-end** do site, a parte visual onde o usuário interage e visualiza as informações presentes.

Por exemplo, podemos pegar o **site da Hashtag**: toda essa parte visual que você vê e com a qual interage é construída com **HTML**, **CSS** e **JavaScript**, sendo o HTML a base entre essas três tecnologias.



É o **HTML** que nos permite ter e estruturar links, botões, imagens, barra de navegação, vídeos e todos os demais elementos da página.

O que é CSS?

CSS significa **Cascading Style Sheets**. Ele é responsável por estilizar os elementos visuais no site. O CSS permite personalizar, formatar e criar o visual de uma página.

Enquanto o HTML estrutura os elementos, o **CSS** define as cores, tamanhos, posições, sombras, fontes, entre outras **edições visuais da página**.

O que é JavaScript?

JavaScript é uma **linguagem de programação** muito utilizada no desenvolvimento web. Ela foi criada em 1995 com a finalidade de tornar a internet dinâmica.

Ou seja, o **JavaScript** é quem proporciona a **interatividade** e o **dinamismo** das páginas web. Ele permite a criação de efeitos, transições e animações de forma fluida e eficiente no site.

O **JavaScript**, junto com **HTML** e **CSS**, compõe as três principais tecnologias do sistema web. Ele é responsável pela inteligência e dinamismo da página, com movimentação, processamento de dados, atualização de informações, reprodução de vídeos, entre outras ações.

Além disso, mesmo tendo sido criado com esse propósito, o **JavaScript** também pode ser utilizado no **back-end** de uma página, atuando no lado do servidor, criando aplicações web, Web APIs, consumindo e atualizando informações de uma base de dados, e assim por diante.

Por que é importante saber HTML?

É claro que o HTML não será importante para todas as aplicações do Python. Porém, em algumas áreas, é muito importante conhecer e entender seu funcionamento e estrutura.

Por exemplo, na área de **automação web**, utilizamos o HTML ao inspecionar uma página para encontrar os elementos com os quais desejamos interagir.

Conhecendo a estrutura de uma página web e como um código HTML funciona, ficará mais **fácil** e **prático** se localizar no código da página e saber qual elemento selecionar.

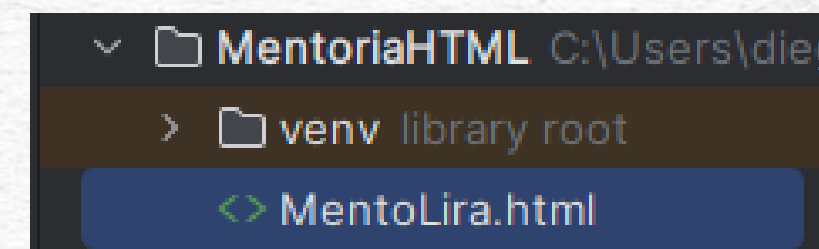
Além disso, para a construção de sites utilizando **Python**, é essencial conhecer **HTML**, **CSS** e **JavaScript**.

O objetivo desta aula será construir a estrutura de uma página HTML, tendo como referência a **página do site da Hashtag**.

Não faremos a parte de **CSS** e **JavaScript**, então ficará algo mais simples, mas o intuito é que você **entenda as principais estruturas do HTML**.

Para esta aula, é recomendado que você use o **PyCharm** ou outro editor que aceite **HTML**, como o **VS Code**.

O primeiro passo será iniciar um novo **projeto no PyCharm** ou no editor que você estiver utilizando e, dentro dele, criar um **arquivo HTML**. Vou chamar meu arquivo de **MentoLira.html**.



Provavelmente, ao abrir esse arquivo, o seu editor de código já montará a **estrutura base de um arquivo HTML**.

```
<> MentoLira.html x
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Title</title>
6  </head>
7  <body>
8
9  </body>
10 </html>
```

Essa base já traz alguns dos elementos principais presentes em um **documento HTML**, organizados dentro de suas respectivas **tags**.

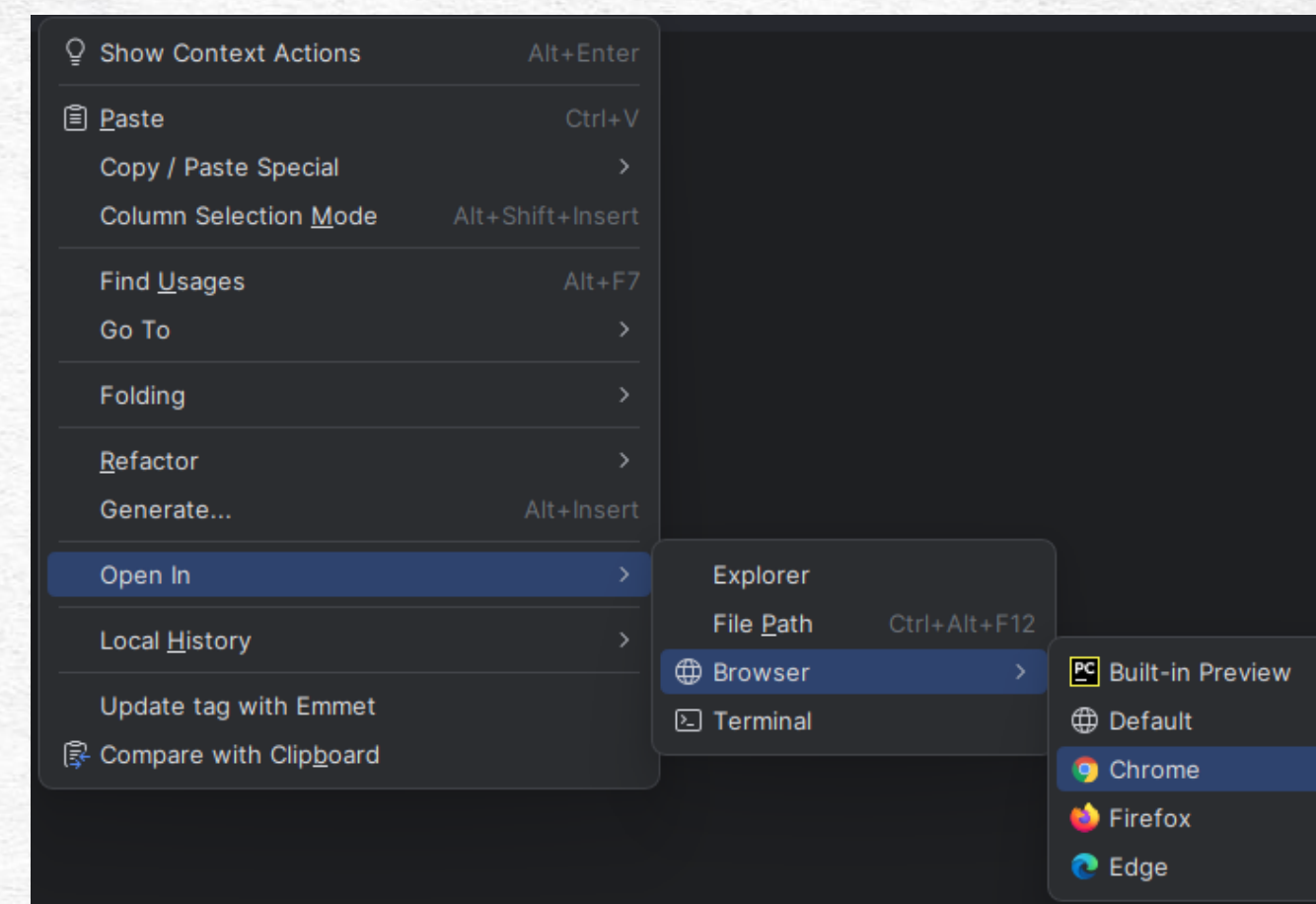
As **tags** são geralmente definidas com uma tag de abertura **<tag>** e uma tag de fechamento **</tag>**. Em alguns casos, temos apenas uma **única tag** que serve tanto para abertura quanto para fechamento do elemento.

A tag **<html>** é responsável por envolver todo o conteúdo da página. É dentro dela que iremos definir a **<head>** e o **<body>** do site.

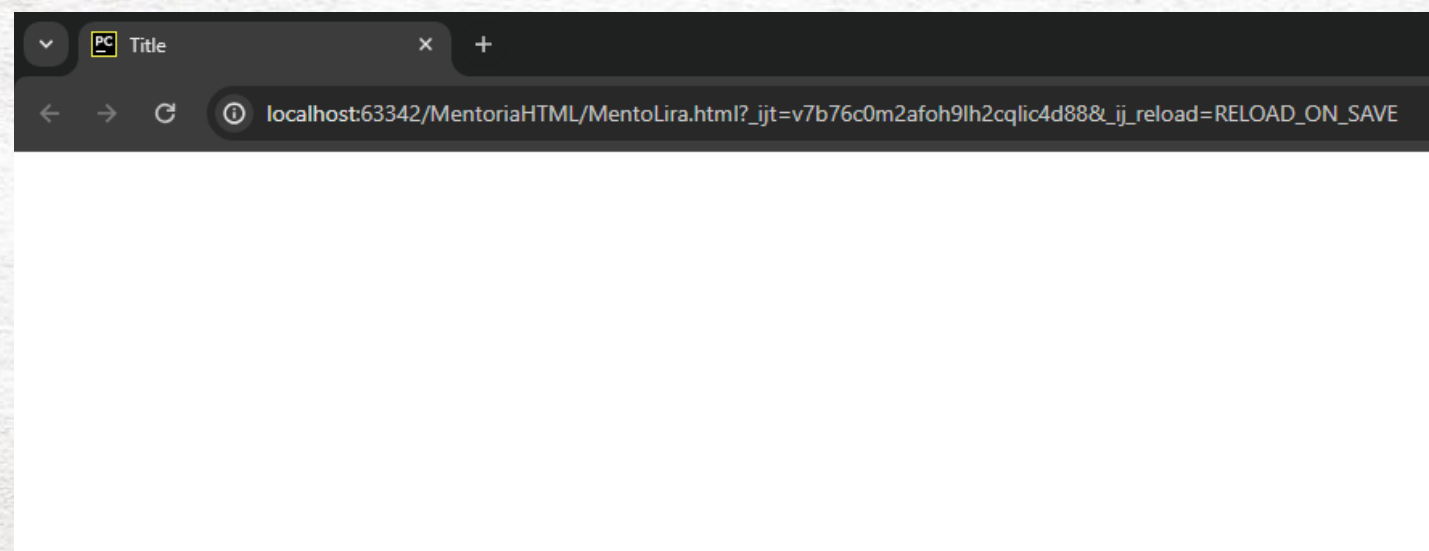
Na **<head>** são inseridas as **informações** sobre o documento, como o título da página, referências de estilo CSS, scripts e outras metainformações. Essas são as informações que serão carregadas primeiro, as configurações da página.

Já o **<body>** é o **corpo** da página, e é nele que fica o **conteúdo principal**, como texto, imagens, links e outros elementos que podemos ver.

No **PyCharm**, é possível visualizar essa página padrão que foi criada, clicando com o botão direito do mouse e selecionando o **navegador** no qual você deseja visualizar como a página está.



Por enquanto, se abrirmos a página em qualquer navegador, ela estará em branco, porque não possui ainda nenhum elemento dentro do seu `<body>`.



Vamos começar editando nosso HTML para construir uma estrutura semelhante à da página da Hashtag.

Começando pelo idioma definido dentro da tag `<html>`, podemos alterar de "en" (inglês) para "pt-BR" (português do Brasil).

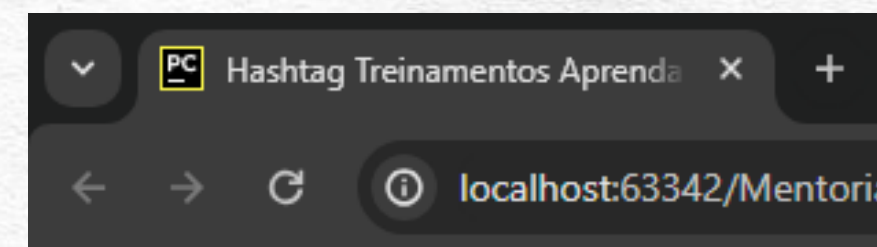
```
<> MentoLira.html x
1  <!DOCTYPE html>
2  <html lang="pt-BR">
```

Feito isso, vamos editar a nossa `<head>`. Dentro dela, temos a tag `<meta>`, onde definimos a formatação e a codificação do texto da página, que, nesse caso, é **UTF-8**.

Em seguida, temos o título do nosso site `<title>`, essa tag define o título que será exibido nas guias do navegador.

```
<> MentoLira.html x
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4      <meta charset="UTF-8">
5      <title>Hashtag Treinamentos Aprenda Tudo de Excel, VBA e Power BI Aqui!</title>
6  </head>
7  <body>
8
9  </body>
10 </html>
```

Visualizando a nossa página, veremos que o título foi modificado no navegador.



Agora que já definimos a `<head>` da página, vamos partir para o `<body>`, o conteúdo do site em si.

O primeiro elemento que chama atenção e se destaca na página da Hashtag é o texto de chamada principal: “**Torne-se uma referência na empresa onde você trabalha ou vai trabalhar com a Hashtag Treinamentos**”.

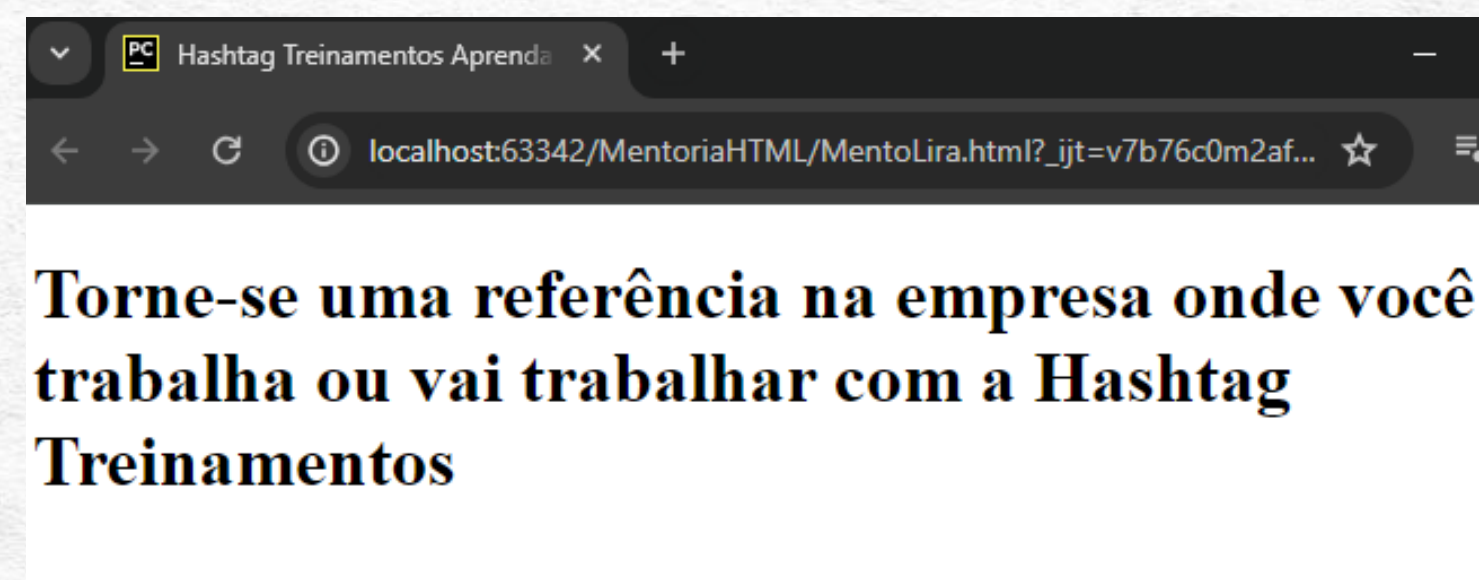


Para adicionar esse elemento à nossa página, vamos criar um **heading** (título) representado pela tag `<h1>`.

Os **headings** variam de tamanho e importância, sendo `<h1>` o mais importante e `<h6>` o menos relevante.

Essa relevância, além de ajudar a estruturar a página do seu site, também é considerada pelos mecanismos de busca.

```
<> MentoLira.html x
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4      <meta charset="UTF-8">
5      <title>Hashtag Treinamentos Aprenda Tudo de Excel, VBA e Power BI
        Aqui!</title>
6  </head>
7  <body>
8      <h1>Torne-se uma referência na empresa onde você trabalha ou vai trabalhar
        com a Hashtag Treinamentos</h1>
9  </body>
10 </html>
```

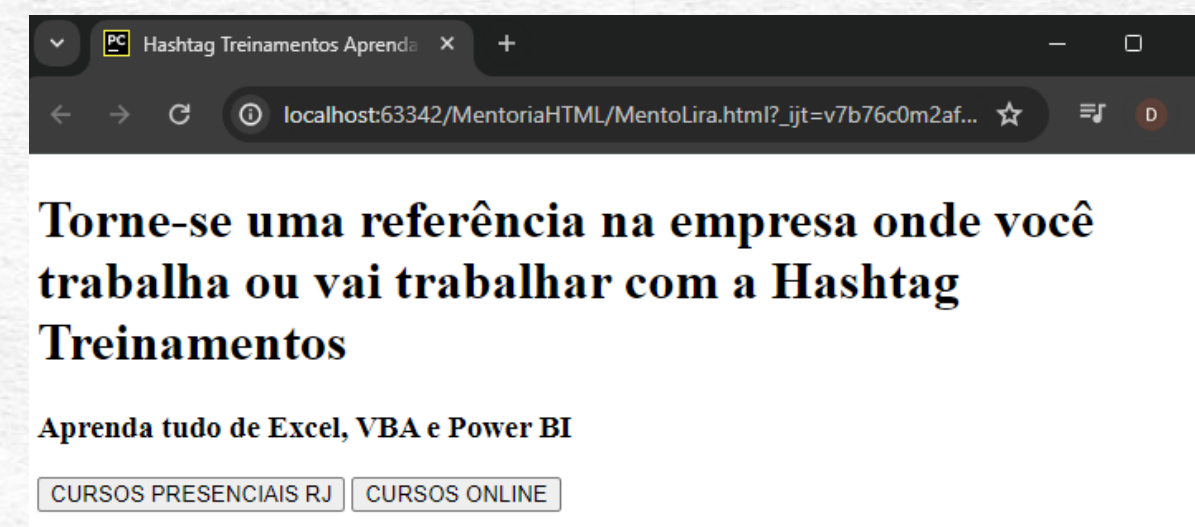
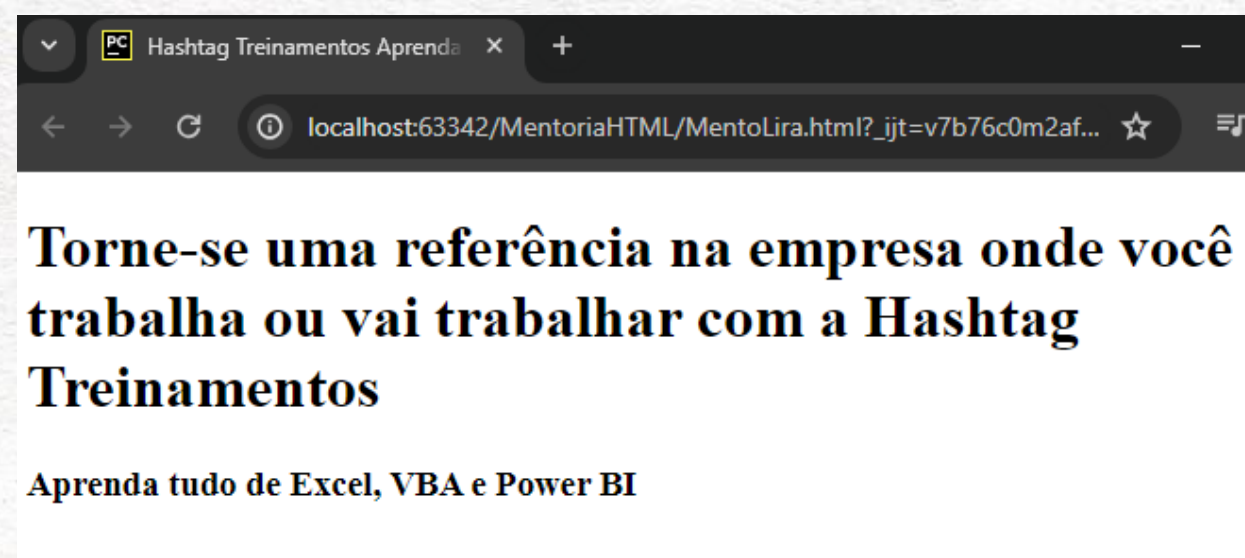


Além desse título, precisamos adicionar um **segundo heading** para a mensagem que complementa o nosso `<h1>`. Como esse é um texto um pouco menor e menos relevante, utilizaremos a tag `<h3>`.

Logo abaixo desses textos, temos na página **dois botões**. A tag responsável por criar botões no HTML é a tag `<button>` e nela definimos o texto que queremos exibir dentro do botão.

```
<> MentoLira.html x
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4      <meta charset="UTF-8">
5      <title>Hashtag Treinamentos Aprenda Tudo de Excel, VBA e Power BI
        Aqui!</title>
6  </head>
7  <body>
8      <h1>Torne-se uma referência na empresa onde você trabalha ou vai trabalhar
        com a Hashtag Treinamentos</h1>
9      <h3>Aprenda tudo de Excel, VBA e Power BI</h3>
10 </body>
11 </html>
```

```
<> MentoLira.html x
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4      <meta charset="UTF-8">
5      <title>Hashtag Treinamentos Aprenda Tudo de Excel, VBA e Power BI
        Aqui!</title>
6  </head>
7  <body>
8      <h1>Torne-se uma referência na empresa onde você trabalha ou vai trabalhar
        com a Hashtag Treinamentos</h1>
9      <h3>Aprenda tudo de Excel, VBA e Power BI</h3>
10 <button>CURSOS PRESENCIAIS RJ</button>
11 <button>CURSOS ONLINE</button>
12 </body>
13 </html>
```



Por enquanto, nossos botões não farão nada; mais adiante iremos adicionar os **links** para os quais eles deverão redirecionar quando clicados.

Agora, precisamos colocar o vídeo na nossa página. Para adicionar um vídeo com HTML, podemos utilizar três tags diferentes: `<video></video>`, `<embed>`, ou `<iframe></iframe>`.

A tag `<video>` é usada especificamente para incorporar vídeos ao seu site. Ela é mais recomendada quando você tem **controle** sobre o arquivo de vídeo que deseja exibir (o arquivo está no seu computador ou no seu servidor).

A tag `<embed>` pode ser usada para incorporar conteúdo de diferentes tipos, incluindo vídeos, PDFs, etc.

Por fim, a tag `<iframe>` é mais utilizada quando desejamos incorporar vídeos de outras fontes, como os vídeos do YouTube.

O próprio YouTube disponibiliza o código completo dentro de um `<iframe>` quando você seleciona a opção "**Incorporar**" dentro do menu de compartilhamento.

Vamos pegar este vídeo da Hashtag como exemplo:
<https://www.youtube.com/watch?v=YIVPnII5138>

Com ele aberto no seu YouTube, clique em **Compartilhar** e depois em **Incorporar**.



Vamos copiar todo esse código.

Colaremos o código dentro do **body** do nosso arquivo HTML.

```
7 <body>
8   <h1>Torne-se uma referência na empresa onde você trabalha ou vai trabalhar com a Hashtag
   Treinamentos</h1>
9   <h3>Aprenda tudo de Excel, VBA e Power BI</h3>
10  <button>CURSOS PRESENCIAIS RJ</button>
11  <button>CURSOS ONLINE</button>
12  <iframe
13    width="560"
14    height="315"
15    src="https://www.youtube.com/embed/YIVPnIL5138?si=t6r7N_Ysj6Ma-R_4"
16    title="YouTube video player"
17    frameborder="0"
18    allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
19    picture-in-picture; web-share" referrerpolicy="strict-origin-when-cross-origin"
20    allowfullscreen>
21  </iframe>
</body>
```

Perceba que o vídeo está sendo exibido na página do site e podemos interagir com ele normalmente.

A tag **<iframe>** é muito útil porque ela basicamente nos permite “colocar um pedaço de um site” dentro do nosso próprio site.

O YouTube utiliza **iframes** como forma de permitir que os vídeos sejam incorporados, porque dessa forma ele ainda consegue ter um certo controle sobre o que pode ou não ser feito com o vídeo.

Além disso, a **fonte do vídeo** ainda é controlada pelo YouTube, então caso o vídeo seja removido do YouTube por violar alguma diretriz da comunidade, o vídeo também será removido da sua página.

Compreender a tag **<iframe>** é importante não só para a criação de sites, mas também para automação de processos web com Python.



Dentro de uma tag `<iframe>`, podemos ter elementos comuns ao site principal onde esse `iframe` está inserido. No entanto, é preciso primeiro acessar esse `iframe` por meio da automação para depois conseguir interagir com os elementos dentro dele.

Com o vídeo inserido na nossa página, concluímos a primeira seção do site. Dentro do **HTML**, temos algumas **tags** que podemos utilizar para separar um elemento do outro.

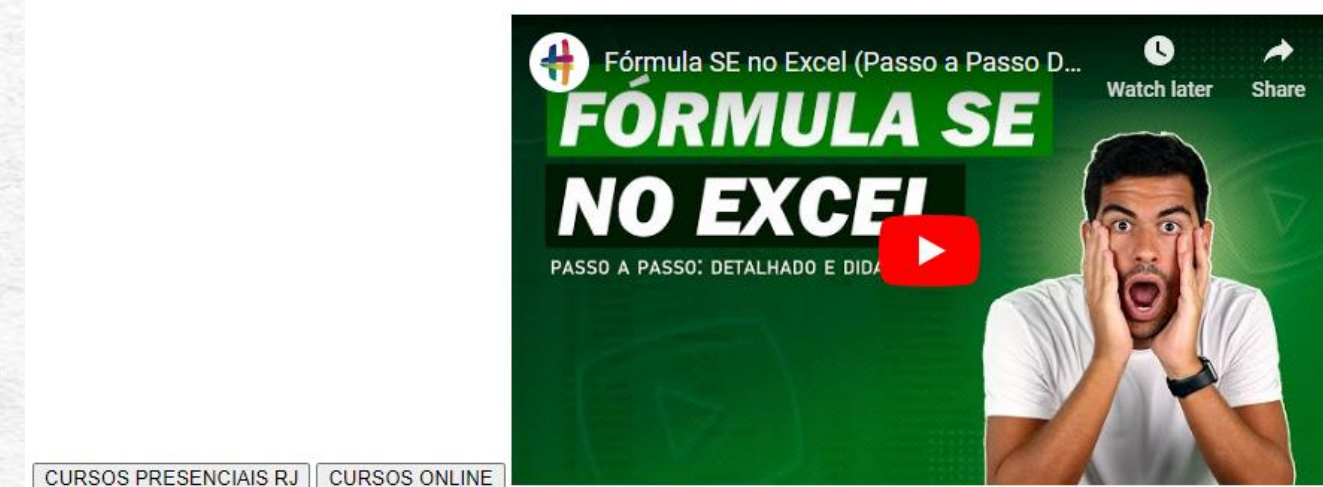
A tag `<br>` é utilizada para adicionar uma quebra de linha, ou seja, ela irá inserir uma linha vazia entre os elementos em que você a definiu.

Além dela, temos a tag `<hr>`. Esta tag cria uma **linha horizontal** na página. Utilizaremos essa tag para **separar** a primeira seção do nosso site da próxima.

```
7 <body>
8   <h1>Torne-se uma referência na empresa onde você trabalha ou vai trabalhar com a Hashtag
   Treinamentos</h1>
9   <h3>Aprenda tudo de Excel, VBA e Power BI</h3>
10  <button>CURSOS PRESENCIAIS RJ</button>
11  <button>CURSOS ONLINE</button>
12  <iframe
13    width="560"
14    height="315"
15    src="https://www.youtube.com/embed/YIVPnIL5138?si=tGr7N_Ysj6Ma-R_4"
16    title="YouTube video player"
17    frameborder="0"
18    allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
    picture-in-picture; web-share" referrerpolicy="strict-origin-when-cross-origin"
19    allowfullscreen>
20  </iframe>
21  <hr>
22 </body>
```

Torne-se uma referência na empresa onde você trabalha ou vai trabalhar com a Hashtag Treinamentos

Aprenda tudo de Excel, VBA e Power BI



Dentro dessa segunda seção, vamos criar a estrutura desta parte do site da Hashtag.



Primeiro, precisamos ter um **título** e, para isso, utilizaremos a tag `<h2>`.

```
21 <hr>
22 <h2>CONHEÇA OS DIFERENCIAIS DA HASHTAG TREINAMENTOS</h2>
```

Logo abaixo dela, precisamos definir os itens que apresentam os diferenciais da Hashtag.

Cada um desses itens será um parágrafo diferente e, para isso, utilizaremos a tag `<p>`. Esta tag é comumente utilizada no HTML quando queremos **definir textos dentro da página**.

Ao todo, teremos **6 parágrafos**, pois cada item terá um parágrafo para o “**título**” e outro para a **descrição**.

```
21 <hr>
22 <h2>CONHEÇA OS DIFERENCIAIS DA HASHTAG TREINAMENTOS</h2>
23 <p>Nível Impressionador</p>
24 <p> Começamos o curso do absoluto ZERO e te levamos até além do avançado, até o nível
    Impressionador. Nesse nível, as suas planilhas e ferramentas impressionam qualquer chefe
    ou recrutador. Quem está no nível Impressionador é a referência na empresa em que
    trabalha quando o assunto é Excel!</p>
25 <p>Foco no Mercado</p>
26 <p> Nosso material é 100% voltado para o Mercado de Trabalho, por isso incluímos diversos
    exercícios e provas de Excel de processos seletivos. Vamos te ensinar tudo o que você
    precisa saber para entrar no mercado de trabalho e se tornar uma referência na empresa
    onde trabalha ou onde vai trabalhar. </p>
27 <p>+21.500 alunos formados</p>
28 <p>Nos últimos anos a Hashtag Treinamentos transformou a vida de milhares de alunos em
    todo o Brasil com seus cursos online e presencial! Garantimos o resultado porque temos
    um método totalmente diferenciado e que já foi comprovado com milhares de cases de
    sucesso.</p>
29 </body>
```



Agora, quero te mostrar duas tags que são muito importantes e úteis: a tag `<div></div>` e a tag `<span></span>`.

A tag `<div>` nos permite agrupar elementos dentro do HTML em uma seção, sem uma semântica específica. Ou seja, ela não possui significado por si só, mas é importante para **organizar e estruturar** o código HTML, agrupando elementos relacionados.

Essa tag é muito útil quando desejamos aplicar o mesmo **estilo e formatação** a uma determinada parte da página.

Podemos englobar toda essa parte dentro de uma `<div>` e aplicar a formatação nela, que será válida para **todos os elementos contidos dentro dela**.

Vamos separar cada uma das seções do nosso site **dentro de uma div**.

```
7 <body>
8   <div>
9     <h1>Torne-se uma referência na empresa onde você trabalha ou vai trabalhar com a
      Hashtag Treinamentos</h1>
10    <h3>Aprenda tudo de Excel, VBA e Power BI</h3>
11    <button>CURSOS PRESENCIAIS RJ</button>
12    <button>CURSOS ONLINE</button>
13    <iframe
14      width="560"
15      height="315"
16      src="https://www.youtube.com/embed/YIVPnIL5138?si=t6r7N_Ysj6Ma-R_4"
17      title="YouTube video player"
18      frameborder="0"
19      allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
      picture-in-picture; web-share"
      referrerpolicy="strict-origin-when-cross-origin"
20      allowfullscreen>
21    </iframe>
22  </div>
23  <hr>
```

```
23  <hr>
24  <div>
25    <h2>CONHEÇA OS DIFERENCIAIS DA HASHTAG TREINAMENTOS</h2>
26    <p>Nível Impressionador</p>
27    <p> Começamos o curso do absoluto ZERO e te levamos até além do avançado, até o nível
      Impressionador. Nesse nível, as suas planilhas e ferramentas impressionam qualquer
      chefe ou recrutador. Quem está no nível Impressionador é a referência na empresa em
      que trabalha quando o assunto é Excel!</p>
28    <p>Foco no Mercado</p>
29    <p> Nosso material é 100% voltado para o Mercado de Trabalho, por isso incluímos
      diversos exercícios e provas de Excel de processos seletivos. Vamos te ensinar tudo
      o que você precisa saber para entrar no mercado de trabalho e se tornar uma
      referência na empresa onde trabalha ou onde vai trabalhar. </p>
30    <p>+21.500 alunos formados</p>
31    <p>Nos últimos anos a Hashtag Treinamentos transformou a vida de milhares de alunos
      em todo o Brasil com seus cursos online e presencial! Garantimos o resultado porque
      temos um método totalmente diferenciado e que já foi comprovado com milhares de
      cases de sucesso.</p>
32  </div>
33 </body>
```

Dentro dessas **divs**, podemos criar outras divs para agrupar e separar outros elementos. Por exemplo, podemos criar uma segunda div para o título e os parágrafos da **segunda seção** da nossa página.

```
24 <div>
25   <div>
26     <h2>CONHEÇA OS DIFERENCIAIS DA HASHTAG TREINAMENTOS</h2>
27   </div>
28   <div>
29     <p>Nível Impressionador</p>
30     <p> Começamos o curso do absoluto ZERO e te levamos até além do avançado, até o
      nível Impressionador. Nesse nível, as suas planilhas e ferramentas impressionam
      qualquer chefe ou recrutador. Quem está no nível Impressionador é a referência
      na empresa em que trabalha quando o assunto é Excel!</p>
31     <p>Foco no Mercado</p>
32     <p> Nosso material é 100% voltado para o Mercado de Trabalho, por isso incluimos
      diversos exercícios e provas de Excel de processos seletivos. Vamos te ensinar
      tudo o que você precisa saber para entrar no mercado de trabalho e se tornar uma
      referência na empresa onde trabalha ou onde vai trabalhar. </p>
33     <p>+21.500 alunos formados</p>
34     <p>Nos últimos anos a Hashtag Treinamentos transformou a vida de milhares de
      alunos em todo o Brasil com seus cursos online e presencial! Garantimos o
      resultado porque temos um método totalmente diferenciado e que já foi comprovado
      com milhares de cases de sucesso.</p>
35   </div>
36 </div>
```

A tag **** é similar à tag **<div>**, mas a diferença está na aplicação.

As **<div>** são mais utilizadas para estruturas maiores, que normalmente ocupam mais de uma linha de código, enquanto a tag **** é utilizada para partes menores do código, envolvendo apenas um ou alguns poucos elementos.

Vamos colocar os títulos dos nossos parágrafos dentro de um ****.

```
28 <div>
29   <span><p>Nível Impressionador</p></span>
30   <p> Começamos o curso do absoluto ZERO e te levamos até além do avançado, até o
      nível Impressionador. Nesse nível, as suas planilhas e ferramentas impressionam
      qualquer chefe ou recrutador. Quem está no nível Impressionador é a referência
      na empresa em que trabalha quando o assunto é Excel!</p>
31   <span><p>Foco no Mercado</p></span>
32   <p> Nosso material é 100% voltado para o Mercado de Trabalho, por isso incluimos
      diversos exercícios e provas de Excel de processos seletivos. Vamos te ensinar
      tudo o que você precisa saber para entrar no mercado de trabalho e se tornar uma
      referência na empresa onde trabalha ou onde vai trabalhar. </p>
33   <span><p>+21.500 alunos formados</p></span>
34   <p>Nos últimos anos a Hashtag Treinamentos transformou a vida de milhares de
      alunos em todo o Brasil com seus cursos online e presencial! Garantimos o
      resultado porque temos um método totalmente diferenciado e que já foi comprovado
      com milhares de cases de sucesso.</p>
35 </div>
```

Você notará que, apesar de termos adicionado todas essas tags ao nosso HTML, o visual da nossa página se mantém o mesmo.

Torne-se uma referência na empresa onde você trabalha ou vai trabalhar com a Hashtag Treinamentos

Aprenda tudo de Excel, VBA e Power BI



CURSOS PRESENCIAIS RJ CURSOS ONLINE

CONHEÇA OS DIFERENCIAIS DA HASHTAG TREINAMENTOS

Nível Impressionador

Começamos o curso do absoluto ZERO e te levamos até além do avançado, até o nível Impressionador. Nesse nível, as suas planilhas e ferramentas impressionam qualquer chefe ou recrutador. Quem está no nível Impressionador é a referência na empresa em que trabalha quando o assunto é Excel!

Foco no Mercado

Agora, criaremos a terceira seção do nosso site. Ela também estará dentro de uma `<div>` e será separada da segunda seção por uma tag `<hr>`.

```
36      <p>Nos últimos anos a Hashtag Treinamentos transformou a vida de milhares de  
37      alunos em todo o Brasil com seus cursos online e presencial! Garantimos o  
38      resultado porque temos um método totalmente diferenciado e que já foi comprovado  
39      com milhares de cases de sucesso.</p>  
40      </div>  
41      <hr>  
42      <div>  
43      </div>  
44      </body>  
45      </html>  
46
```

Dentro dessa parte, teremos duas imagens mostrando as aparições da Hashtag na mídia.



Primeiro, vamos definir nosso `<h2>`, que é o título dessa seção, e em seguida adicionaremos as duas imagens referentes à **Globo News** e ao jornal **O Globo**.

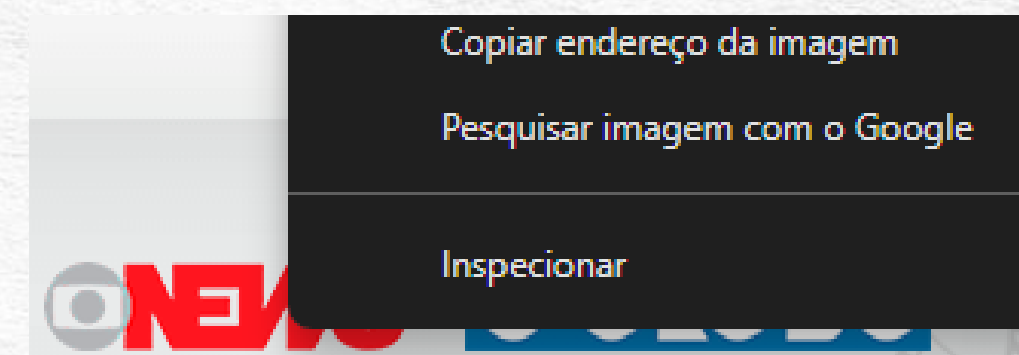
Para inserir uma imagem no código HTML, utilizamos a tag `<img>` com o parâmetro `src`, que define o endereço/caminho da imagem.

Você já deve ter reparado que algumas tags possuem dentro delas parâmetros aos quais atribuímos um valor.

Esses parâmetros podem ser **obrigatórios** ou **opcionais** e permitem aplicar formatações e funcionalidades a uma determinada **tag**. Um parâmetro sempre será declarado dentro da **tag de abertura**.

No caso da tag `<img>`, o parâmetro `src` é obrigatório. É ele que informará o caminho para que a imagem seja exibida. O ideal é que esse caminho seja o **endereço para o seu servidor** onde a imagem estará hospedada.

Para o nosso exemplo, eu vou apenas copiar o endereço da imagem a partir do site da Hashtag.



Faremos isso para as duas imagens e, para cada tag `<img>`, atribuiremos um dos caminhos como valor para o parâmetro `src`.

```
40 <hr>
41
42 <div>
43   <h2>HASHTAG TREINAMENTOS NA MÍDIA</h2>
44   
45   
46 </div>
```

Observe também que a tag de imagem não requer uma **tag de fechamento**.

HASHTAG TREINAMENTOS NA MÍDIA



Feito isso, vamos criar a última seção do nosso site referente aos conteúdos gratuitos.

CONTEÚDOS GRATUITOS DA HASHTAG TREINAMENTOS

[BLOG HASHTAG](#)[YOUTUBE](#)[INSTAGRAM](#)

Esta parte, ao invés de seguir a estrutura da página da Hashtag, que tem cada conteúdo em um botão, vamos criar na **forma de listas**, para que você possa compreender também como criar uma **lista no HTML**.

Então vamos usar a tag **<hr>** para separar essa seção da anterior e definir a **div** onde iremos adicionar a nossa lista.

```
41
42     <div>
43         <h2>HASHTAG TREINAMENTOS NA MÍDIA</h2>
44         
46         
48     </div>
49
50     <hr>
51
52     <div>
53     </div>
54 </body>
```

Dentro dessa **div** teremos um título **<h2>**, como nas outras seções, e criaremos a lista em si.

Para criar uma lista em HTML temos duas tags possíveis: a tag **** (unordered list) e a tag **** (ordered list).

A `<ul>` é utilizada para criar uma **lista não ordenada**, onde os itens não possuem uma sequência específica. Cada item da lista é identificado por um ponto ou outro símbolo que pode ser definido pelo CSS.

A `<ol>`, por sua vez, é usada para criar uma **lista ordenada**, onde os itens são numerados ou têm uma sequência específica. Cada item da lista é marcado com um número ou letra, indicando sua posição na lista.

Além dessas tags, precisamos declarar dentro delas a tag `<li>` (**list item**). Cada tag `<li>` é utilizada para definir um item da lista, seja ela ordenada ou não.

Vamos criar a nossa lista como uma lista não ordenada.

```
48 <hr>
49
50 <div>
51   <h2>CONTEÚDOS GRATUITOS DA HASHTAG TREINAMENTOS</h2>
52   <ul>
53     <li>Blog Hashtag</li>
54     <li>Youtube</li>
55     <li>Instagram</li>
56   </ul>
57 </div>
58 </body>
59 </html>
```

CONTEÚDOS GRATUITOS DA HASHTAG TREINAMENTOS

- Blog Hashtag
- Youtube
- Instagram

Porém, a nossa lista, assim como outros elementos dentro do nosso site, deve ter a capacidade de nos direcionar para outras páginas na internet. Para fazer isso, precisamos utilizar a **tag de âncora** `<a>`.

Sempre que você quiser criar um **link** dentro do seu site, você coloca o elemento que você deseja que se comporte como um link **dentro da tag** `<a>`. Essas tags possuem o atributo **href**, que recebe como valor o **link de destino** que aquela tag de âncora levará o usuário.

Então, para que um item da nossa lista seja um **link**, precisamos envolver todo o item da lista por uma **tag de âncora**. Vamos aplicar isso à nossa lista.

```
50 <div>
51   <h2>CONTEÚDOS GRATUITOS DA HASHTAG TREINAMENTOS</h2>
52   <ul>
53     <a href="https://www.hashtagtreinamentos.com/blog"><li>Blog Hashtag</li></a>
54     <a href="https://www.youtube.com/@HashtagProgramacao"><li>Youtube</li></a>
55     <a href="https://www.instagram.com/hashtagtreinamentos/"><li>Instagram</li></a>
56   </ul>
57 </div>
```

CONTEÚDOS GRATUITOS DA HASHTAG TREINAMENTOS

- [Blog Hashtag](#)
- [Youtube](#)
- [Instagram](#)

Agora nossa lista encaminhará o usuário para o **Blog**, **YouTube** e **Instagram da Hashtag**, dependendo de em qual item o usuário clicar.

Vamos fazer o mesmo procedimento para os nossos **botões** na primeira seção do site.

```
7 <body>
8   <div>
9     <h1>Torne-se uma referência na empresa onde você trabalha ou vai trabalhar com
10    a Hashtag Treinamentos</h1>
11    <h3>Aprenda tudo de Excel, VBA e Power BI</h3>
12    <a href="https://www.hashtagtreinamentos.com/cursos-excel-rj/"><button>CURSOS
    PRESENCIAIS RJ</button></a>
    <a href="https://www.hashtagtreinamentos.com/curso-python"><button>CURSOS
    ONLINE</button></a>
```

Feito isso, podemos partir para a última tag que eu quero te mostrar nessa aula, que é a **tag de formulário**. Ela nos permite criar campos de preenchimento como esse na página da Hashtag.

CADASTRE-SE PARA RECEBER O MINICURSO BÁSICO DE 16 AULAS

Seu primeiro nome*	Seu melhor e-mail*	Enviar

Vamos criar uma nova seção para inserir um formulário como esse na nossa página. Então, assim como fizemos para as outras seções, vamos separá-la utilizando uma tag **<hr>** e criar uma **div** onde iremos criar o formulário.

A tag que nos permite inserir um formulário na nossa página é a tag `<form>`. Geralmente, os campos do formulário são compostos por um texto (`<p>`), um campo de preenchimento (`<input>`) e um botão (`<button>`).

O texto e o botão nós já vimos como criar. O campo de preenchimento que ainda não vimos será uma **tag única** em que podemos passar dentro dela o parâmetro **placeholder** com o texto que desejamos exibir no campo do formulário até que o usuário o preencha manualmente.

Vamos replicar o formulário que temos na página:

```
59      <hr>
60
61      <div>
62          <form>
63              <p>Seu primeiro nome</p>
64              <input placeholder="Seu primeiro nome" />
65              <p>Seu melhor e-mail</p>
66              <input placeholder="Seu melhor e-mail" />
67              <button>Enviar</button>
68          </form>
69      </div>
```

Seu primeiro nome

Seu melhor e-mail

Um ponto importante é que essa é a **estrutura do formulário** dentro do HTML.

Para que ele de fato **funcione**, pegando as informações do usuário e enviando para o banco de dados, é necessário um sistema **back-end** que fará essa ligação entre a página e o banco de dados. Isso pode ser feito com Python ou JavaScript, por exemplo.

Parâmetros Importantes:

Ao longo desta aula, vimos alguns **parâmetros importantes** e suas funções dentro das tags no HTML, como o **src** e o **href**. Além deles, temos dois parâmetros que são essenciais: **class** e **id**.

O parâmetro **id** geralmente é atribuído a um elemento e é específico daquele elemento. Nenhum outro elemento no site terá o mesmo **id** que outro. Por exemplo, nossa **div** com a lista pode receber o **id contgrat**, referente a conteúdos gratuitos.

```
50 <div id="contgrat">
51   <h2>CONTEÚDOS GRATUITOS DA HASHTAG TREINAMENTOS</h2>
52   <ul>
53     <a href="https://www.hashtagtreinamentos.com/blog"><li>Blog Hashtag</li></a>
54     <a href="https://www.youtube.com/@HashtagProgramacao"><li>Youtube</li></a>
55     <a href="https://www.instagram.com/hashtagtreinamentos/"><li>Instagram</li></a>
56   </ul>
57 </div>
```

O **id** será uma forma de identificar esta e apenas esta **div**.

Já o parâmetro **class** é usado para atribuir uma ou mais classes a um elemento. Diferente do **id**, vários elementos podem compartilhar o mesmo valor de **class**. Um elemento com mais de uma classe terá suas classes separadas por um espaço apenas.

```
7 <body>
8   <div class="lira hashtag">
9     <h1>Torne-se uma referência na empresa onde você trabalha ou vai trabalhar com
      a Hashtag Treinamentos</h1>
```

```
26 <div>
27   <div class="lira">
28     <h2>CONHEÇA OS DIFERENCIAIS DA HASHTAG TREINAMENTOS</h2>
29   </div>
30 </div>
```

Podemos usar o **id** e a **class** dentro do **CSS** para aplicar estilos específicos com base na classe ou no **id** dos elementos, e dentro do **JavaScript** para manipular ou acessar esses elementos.

Vamos aplicar um exemplo com **CSS**. Para isso, utilizaremos o **CSS Interno**, que é quando criamos a estilização com CSS diretamente dentro da **<head>** do documento HTML, criando uma tag **<style>**.

Parâmetros Importantes:

Existem diversas **formatações** e **estilizações** que podemos aplicar aos elementos. Para selecionar uma **classe** dentro do CSS, utilizamos a sintaxe **.nome_da_classe**, e o **id** pode ser selecionado a partir da sintaxe **#nome_do_id**.

```
3 <head>
4   <meta charset="UTF-8">
5   <title>Hashtag Treinamentos Aprenda Tudo de Excel, VBA e Power BI Aqui!</title>
6   <style>
7     .lira {
8       color: green;
9       text-align: center;
10    }
11
12    .hashtag {
13      font-family: helvetica;
14    }
15
16    #contgrat{
17      color: red;
18    }
19  </style>
20 </head>
```

Com isso, os elementos que possuem a classe **lira** ficarão com a cor da fonte verde e alinhados ao centro.

Os que possuem a classe **hashtag** terão a família da fonte definida como helvética. E o elemento com o id **contgrat** terá a cor em vermelho.

Torne-se uma referência na empresa onde você trabalha ou vai trabalhar com a Hashtag Treinamentos

Aprenda tudo de Excel, VBA e Power BI



CURSOS PRESENCIAIS RJ CURSOS ONLINE

CONHEÇA OS DIFERENCIAIS DA HASHTAG TREINAMENTOS

Nível Impressionador

Começamos o curso do absoluto ZERO e te levamos até além do avançado, até o nível Impressionador. Nesse nível, as suas planilhas e ferramentas

CONTEÚDOS GRATUITOS DA HASHTAG TREINAMENTOS

- [Blog Hashtag](#)
- [Youtube](#)
- [Instagram](#)



Parâmetros Importantes:

Outra forma de estilizar os elementos com **CSS** é através da forma **inline**, em que adicionamos dentro de cada tag o atributo **style** e passamos os atributos que desejamos alterar.

```
57 <div>
58   <h2 style="color: blue; text-align: center; font-family: arial">HASHTAG
    TREINAMENTOS NA MÍDIA</h2>
59   
60   
61 </div>
```

HASHTAG TREINAMENTOS NA MÍDIA



Além disso, apesar de não abordarmos nenhum **script** em **JavaScript** como exemplo nessa aula, para linkar um código **JavaScript** com nosso documento HTML, basta adicionar a tag **<script>** dentro da tag **<head>**.

Esta tag receberia como argumento o **src** com o caminho para o arquivo JavaScript.

```
19 </style>
20 <script></script>
21 </head>
```

Nessa aula, você aprendeu mais sobre a **estrutura de uma página HTML**, as tags e os parâmetros utilizados. Existem diversas outras tags e parâmetros que podem ser usados dentro de um arquivo HTML.

É claro que um site criado apenas em um arquivo HTML não terá o visual e a interação que um site completo com **CSS** e **JavaScript** teria, mas o objetivo era apenas que você conhecesse mais sobre o HTML e as tags presentes nele.

Como sugestão de material extra, eu indico o site da [W3 Schools](https://www.w3schools.com/html/), onde há uma aba apenas para HTML explicando com exemplos das diferentes funcionalidades, tags e parâmetros que você pode utilizar.



Quando trabalhamos na construção de um site, podemos dividir seu desenvolvimento em **front-end** e **back-end**.

O **front-end** é responsável pela parte visual do site, onde o usuário interage e visualiza as informações contidas na página.

Já o **back-end** cuida da lógica do site, definindo o que acontece quando o usuário interage, além de controlar o envio e recebimento de informações com o banco de dados.

No desenvolvimento web, o **Python** é aplicado principalmente na construção do back-end de um site. Com ele, construímos a lógica de funcionamento, sistemas de autenticação, gerenciamos e interagimos com o banco de dados, entre outras funcionalidades.

Em **Python**, dois frameworks populares são frequentemente utilizados para isso: **Django** e **Flask**, que oferecem uma série de ferramentas e funcionalidades prontas para a criação do site e integração entre o front-end e o back-end.



Essencialmente, a parte visual do site, a estrutura da página, os elementos e edições visuais, as interações e animações são construídas utilizando **HTML**, **CSS** e **JavaScript**.

É pensando nesse contexto que o **PyScript** foi criado. Desenvolvido pela equipe do **Anaconda**, o **PyScript** tem como objetivo facilitar a integração do Python com o front-end, permitindo escrever e executar scripts em Python diretamente em uma página HTML.

A proposta do **PyScript** é possibilitar a construção de um site completo e responsivo apenas utilizando **Python**, **HTML** e **CSS**. Isso inclui a capacidade de integrar códigos e até bibliotecas Python diretamente no front-end do seu site, rodando-os diretamente do navegador.



PyScript is a platform for Python in the browser.

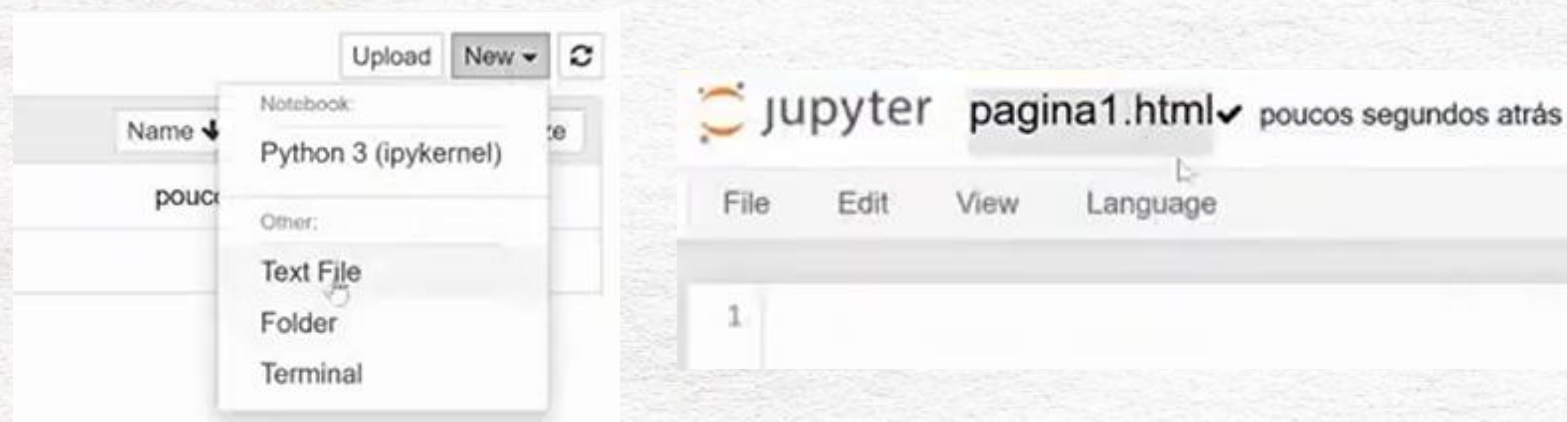
Apesar de parecer uma alternativa ao **JavaScript**, o **PyScript** oferece formas de **interagir com Javascript**, permitindo que os desenvolvedores combinem o potencial das duas linguagens.

Para que os navegadores possam executar códigos e scripts em Python, o **PyScript** utiliza o **Pyodide**, uma versão do Python compatível com **WebAssembly**, que pode ser executada em qualquer navegador.

Você pode conferir as funcionalidades do **PyScript**, exemplos de aplicação e a documentação completa acessando o [site oficial](#) dele.

Ao longo desta aula, eu utilizarei o **Jupyter Notebook**, mas você pode optar pelo editor de códigos de sua preferência.

Dentro do Jupyter Notebook, em vez de criarmos um arquivo Python notebook (.ipynb) como de costume, criaremos um arquivo de texto chamado **pagina1.html**.



Ou seja, vamos utilizar o **Jupyter Notebook** para escrever um **código HTML**. Mas você pode fazer isso em outros editores de código sem problemas.

Na aula anterior, vimos o funcionamento e a construção completa de uma **página HTML**, que é basicamente composta por **tags** que estruturam os elementos visuais dela.

Essa página possui duas estruturas principais: o **<head>** e o **<body>**.

No **<head>**, são inseridas as informações sobre o documento, como o título da página, referências de estilo CSS, scripts e outras metainformações. Essas são as informações que serão carregadas primeiro, as configurações da página.

Já o **<body>** é o corpo da página, e é nele que fica o conteúdo principal, como texto, imagens, links e outros elementos que podemos ver.

Vamos definir a estrutura base da nossa página dentro do arquivo **pagina1.html**.

```
1 <html>
2   <head>
3
4   </head>
5
6   <body>
7
8   </body>
9 </html>
```

Como nós queremos executar códigos em Python dentro dessa página, precisamos apenas copiar da **documentação do PyScript** as tags `<link>` e `<script>`. Dessa forma, estaremos “importando” o PyScript para ser executado na nossa página.

```
1 <html>
2   <head>
3     <link rel="stylesheet" href="https://pyscript.net/alpha/pyscript.css" />
4     <script defer src="https://pyscript.net/alpha/pyscript.js"></script>
5   </head>
6
7   <body>
8
9   </body>
10 </html>
```

O **PyScript** permite a utilização de códigos Python em HTML por meio de três diferentes tags: `<py-env>`, `<py-repl>` e `<py-script>`.

A tag `<py-env>` é utilizada para especificar o ambiente Python que será carregado na página. É através dela que definimos os pacotes e bibliotecas que serão usados na página, como se “instalássemos” as bibliotecas através dela.

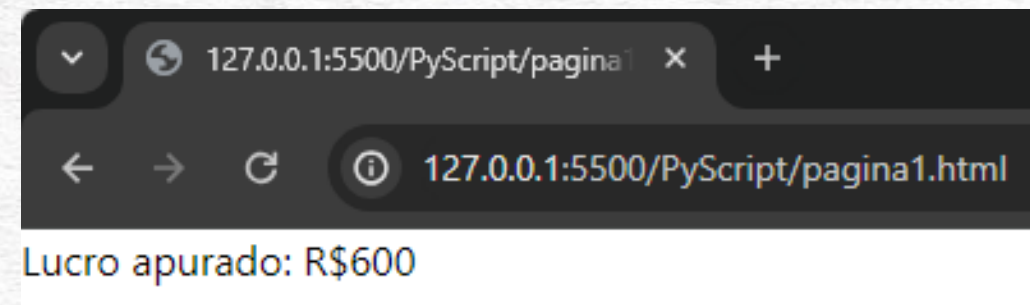
A tag `<py-repl>` cria um **REPL** (Read-Eval-Print Loop) interativo na página. Na prática, isso nos permite escrever e executar códigos Python diretamente do navegador, em um ambiente interativo similar ao prompt do Python.

Por fim, a tag `<py-script>` é utilizada para incluir e executar scripts Python diretamente no HTML. É através dela que o código Python será executado quando a página for carregada.

Vamos começar pela tag `<py-script>`, a mais simples e direta das três estruturas. Podemos utilizá-la dentro do `<body>` da página para escrever **códigos em Python** como já estamos acostumados a fazer.

```
1 <html>
2   <head>
3     <link rel="stylesheet" href="https://pyscript.net/alpha/pyscript.css" />
4     <script defer src="https://pyscript.net/alpha/pyscript.js"></script>
5   </head>
6
7   <body>
8     <py-script>
9       faturamento = 1000
10      custo = 400
11      lucro = faturamento - custo
12      print(f"Lucro apurado: R${lucro}")
13    </py-script>
14  </body>
15 </html>
```

Se você salvar e abrir esse arquivo no seu navegador, verá a mensagem **Lucro apurado: R\$600**.

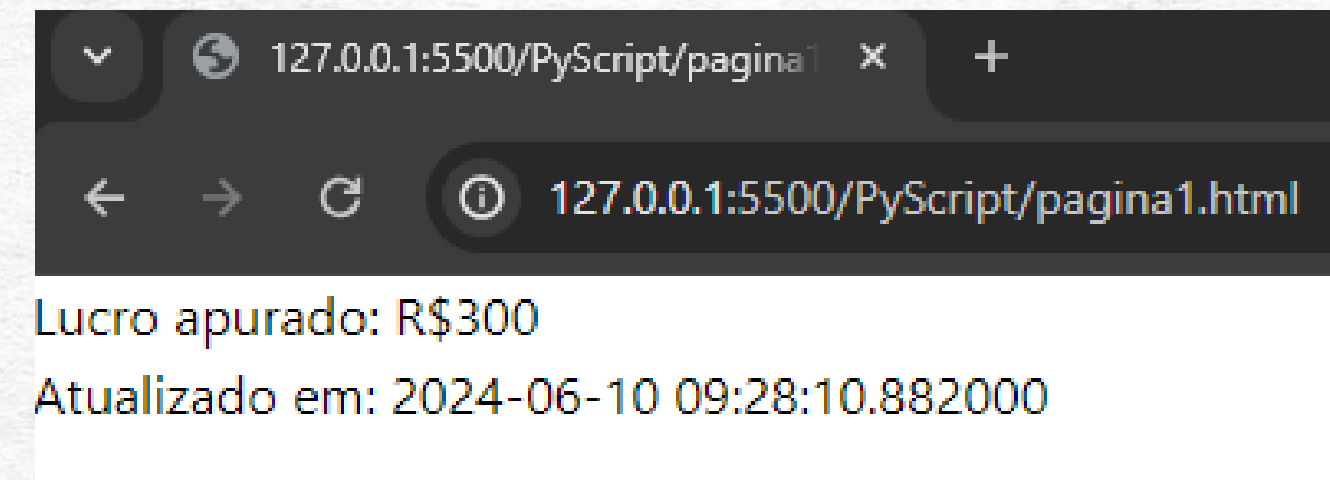


Ou seja, todo o código Python foi executado diretamente do navegador, realizando o cálculo com as variáveis e exibindo o resultado com a função print.

Com o PyScript, independentemente de a pessoa ter o Python instalado no computador, o código em Python será executado, pois isso é feito diretamente no navegador, utilizando uma linguagem que qualquer navegador consegue executar.

É possível até mesmo **importar as bibliotecas** que vêm instaladas por padrão com o Python, **sem precisar instalar nada**, como é o caso da **biblioteca datetime**.

```
1 <html>
2   <head>
3     <link rel="stylesheet" href="https://pyscript.net/alpha/pyscript.css" />
4     <script defer src="https://pyscript.net/alpha/pyscript.js"></script>
5   </head>
6
7   <body>
8     <py-script>
9       from datetime import datetime
10      faturamento = 1000
11      custo = 700
12      lucro = faturamento - custo
13      data_atualizacao = datetime.now()
14      print(f"Lucro apurado: R${lucro}")
15      print(f"Atualizado em: {data_atualizacao}")
16    </py-script>
17  </body>
18 </html>
```

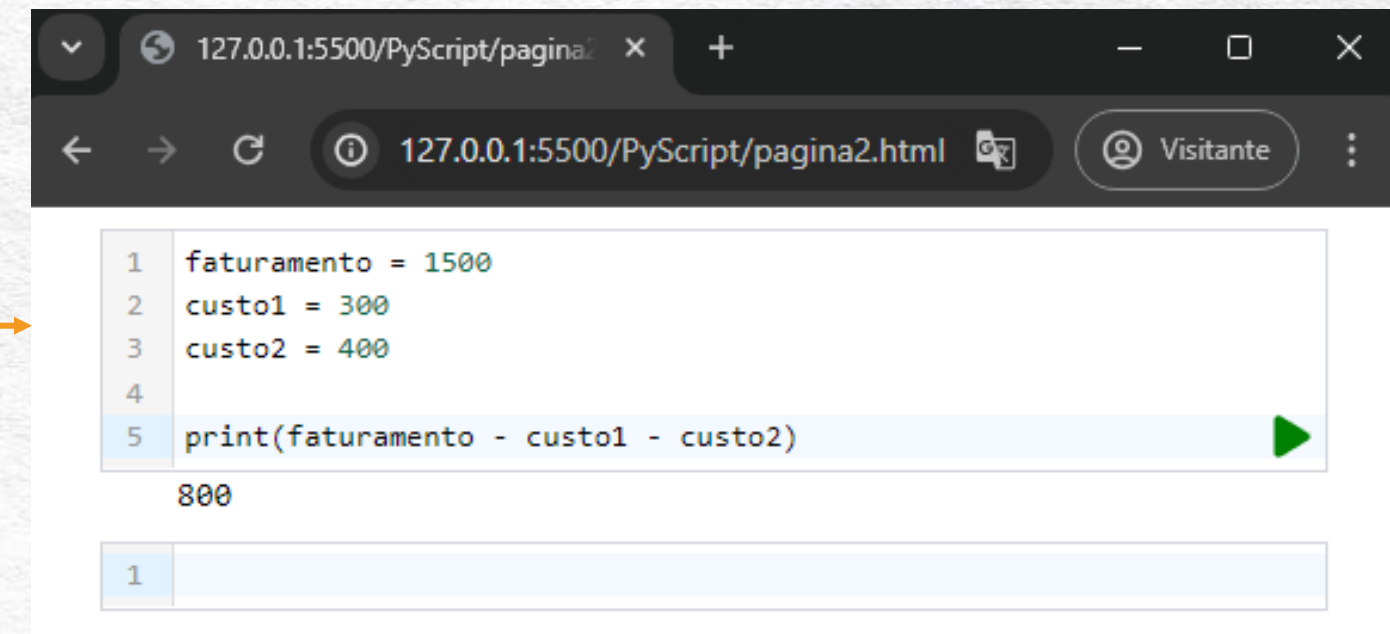


Agora veremos o funcionamento da tag `<py-repl>`. Crie um segundo arquivo chamado `pagina2.html`. A estrutura dessa página será a mesma da anterior, porém dentro do `<body>` dela definiremos a tag `<py-repl>`.

Dentro da tag de abertura, definiremos dois parâmetros: `id="my-repl"` e `auto-generate="true"`.

```
1 <html>
2   <head>
3     <link rel="stylesheet" href="https://pyscript.net/alpha/pyscript.css" />
4     <script defer src="https://pyscript.net/alpha/pyscript.js"></script>
5   </head>
6
7   <body>
8     <py-repl id="my-repl" auto-generate="true"> </py-repl>
9   </body>
10 </html>
```

Abrindo essa página no navegador, teremos uma estrutura semelhante ao Jupyter Notebook que nos permitirá escrever e executar códigos em Python.



Mesmo que a pessoa não tenha o Python instalado, ela conseguirá executar essa página no computador através do navegador, bastando estar conectada à internet.

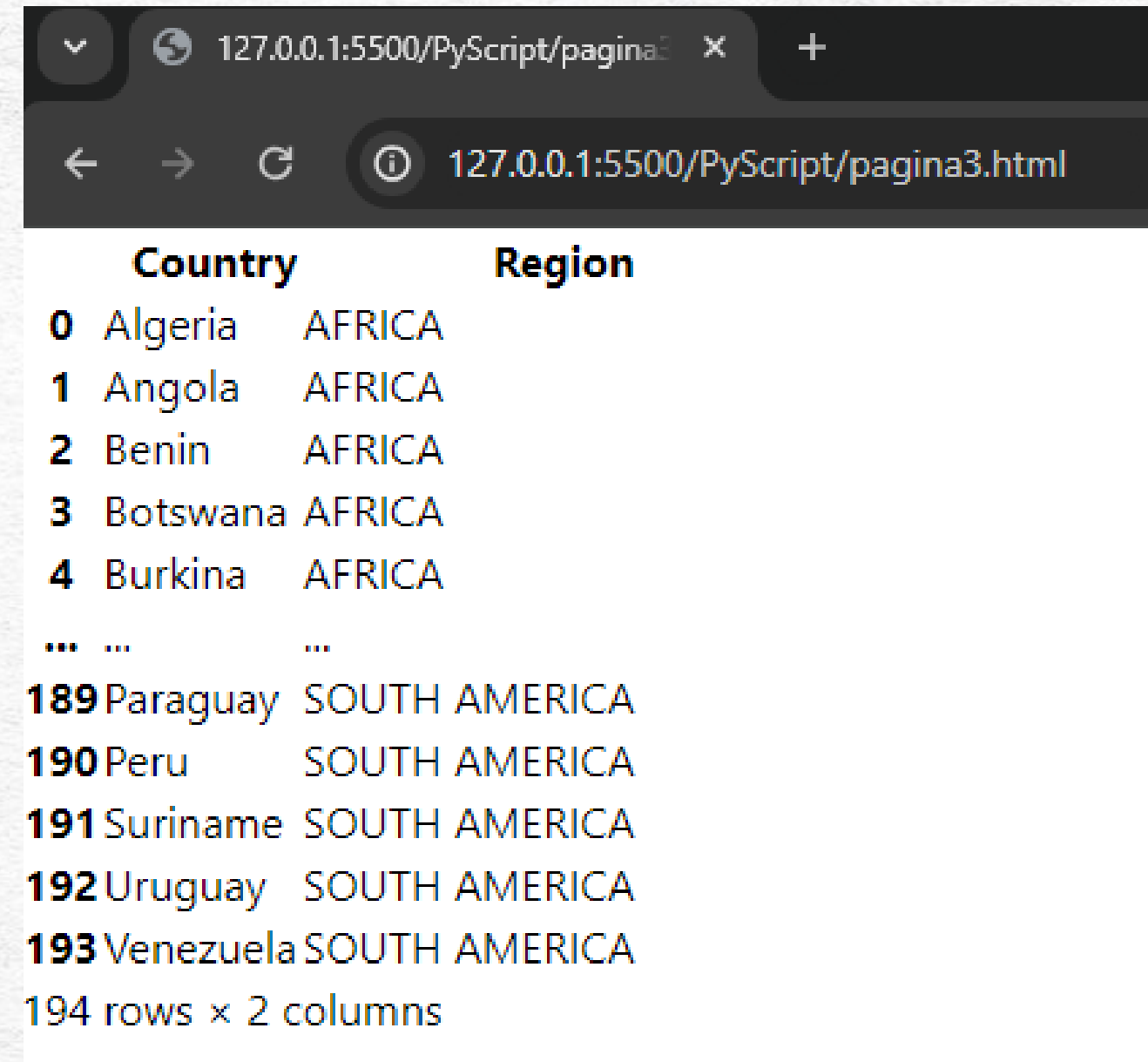
Observação: É importante notar que, com apenas essa estrutura, ao atualizar a página, as informações e os códigos escritos dentro dela serão perdidos. Seriam necessárias outras configurações para manter as informações inseridas após fechar ou atualizar a página.

Feito isso, vamos criar um terceiro arquivo para ver o funcionamento da estrutura **<py-env>**, que nos permitirá utilizar e importar bibliotecas Python que necessitam de instalação, como, por exemplo, o **Pandas**.

A tag **<py-env>** é declarada dentro do **<head>** da página, e dentro dela podemos “instalar” as bibliotecas que queremos utilizar dentro do HTML. A sintaxe para isso será sempre um **hífen seguido do nome da biblioteca**.

Como queremos trabalhar com o **Pandas** e carregar um arquivo CSV a partir de um link, dentro do **<py-script>** no corpo da página, também precisaremos importar, além do Pandas, a funcionalidade **open_url** do **pyodide.http**.

```
1 <html>
2 <head>
3   <link rel="stylesheet" href="https://pyscript.net/alpha/pyscript.css" />
4   <script defer src="https://pyscript.net/alpha/pyscript.js"></script>
5   <py-env>
6     - pandas
7   </py-env>
8 </head>
9
10 <body>
11   <py-script>
12     import pandas as pd
13     from pyodide.http import open_url
14     link = open_url("https://raw.githubusercontent.com/cs109/2014_data/master/countries.csv")
15     tabela = pd.read_csv(link)
16     tabela
17   </py-script>
18 </body>
19 </html>
```



	Country	Region
0	Algeria	AFRICA
1	Angola	AFRICA
2	Benin	AFRICA
3	Botswana	AFRICA
4	Burkina	AFRICA
...	...	...
189	Paraguay	SOUTH AMERICA
190	Peru	SOUTH AMERICA
191	Suriname	SOUTH AMERICA
192	Uruguay	SOUTH AMERICA
193	Venezuela	SOUTH AMERICA
194 rows x 2 columns		

Uma aplicação interessante do **PyScript** é utilizá-lo para compartilhar gráficos de análises de dados com outras pessoas.

Vamos pegar o nosso exemplo anterior e, a partir dele, criar um gráfico simples com **Matplotlib**, utilizando a contagem de valores (**value_counts()**) por região da nossa tabela.

A princípio, você pode pensar que só precisaríamos importar o **Matplotlib** dentro da tag **<py-env>** e executar o código normalmente.

```
1 <html>
2   <head>
3     <link rel="stylesheet" href="https://pyscript.net/alpha/pyscript.css" />
4     <script defer src="https://pyscript.net/alpha/pyscript.js"></script>
5     <py-env>
6       - pandas
7       - matplotlib
8     </py-env>
9   </head>
10
11   <body>
12     <py-script>
13       import pandas as pd
14       import matplotlib.pyplot as plt
15
16       from pyodide.http import open_url
17       link = open_url("https://raw.githubusercontent.com/cs109/2014_data/master/countries.csv")
18       tabela = pd.read_csv(link)
19       tabela
20
21       contagem = tabela["Region"].value_counts()
22       fig, eixo = plt.subplots()
23       contagem.plot(ax=eixo)
24     </py-script>
25   </body>
26 </html>
```

No entanto, se fizermos isso, teremos como resultado uma **página em branco**. Isso acontece porque o **PyScript** não consegue exibir tanto a tabela quanto o gráfico ao mesmo tempo apenas executando-os dentro da tag `<py-script>`.

Para resolver isso, podemos utilizar uma outra funcionalidade do **PyScript** que nos permite editar elementos já existentes no nosso site: o método **write()**. Esse método recebe como argumentos o **id do elemento HTML** e o **conteúdo** que desejamos exibir.

Para que nossa página mostre tanto a tabela quanto o gráfico, precisamos primeiro criar os elementos HTML que irão comportar essas duas informações. Faremos isso **criando duas divs**, uma com o **id tabela_html** e outra com o **id grafico_html**.

```
<html>
  <head>
    <link rel="stylesheet" href="https://pyscript.net/alpha/pyscript.css" />
    <script defer src="https://pyscript.net/alpha/pyscript.js"></script>
    <py-env>
      - pandas
      - matplotlib
    </py-env>
  </head>

  <body>
    <h1>Importando biblioteca usando pyenv</h1>

    <div id="tabela_paises"></div>
    <div id="grafico_qtde"></div>

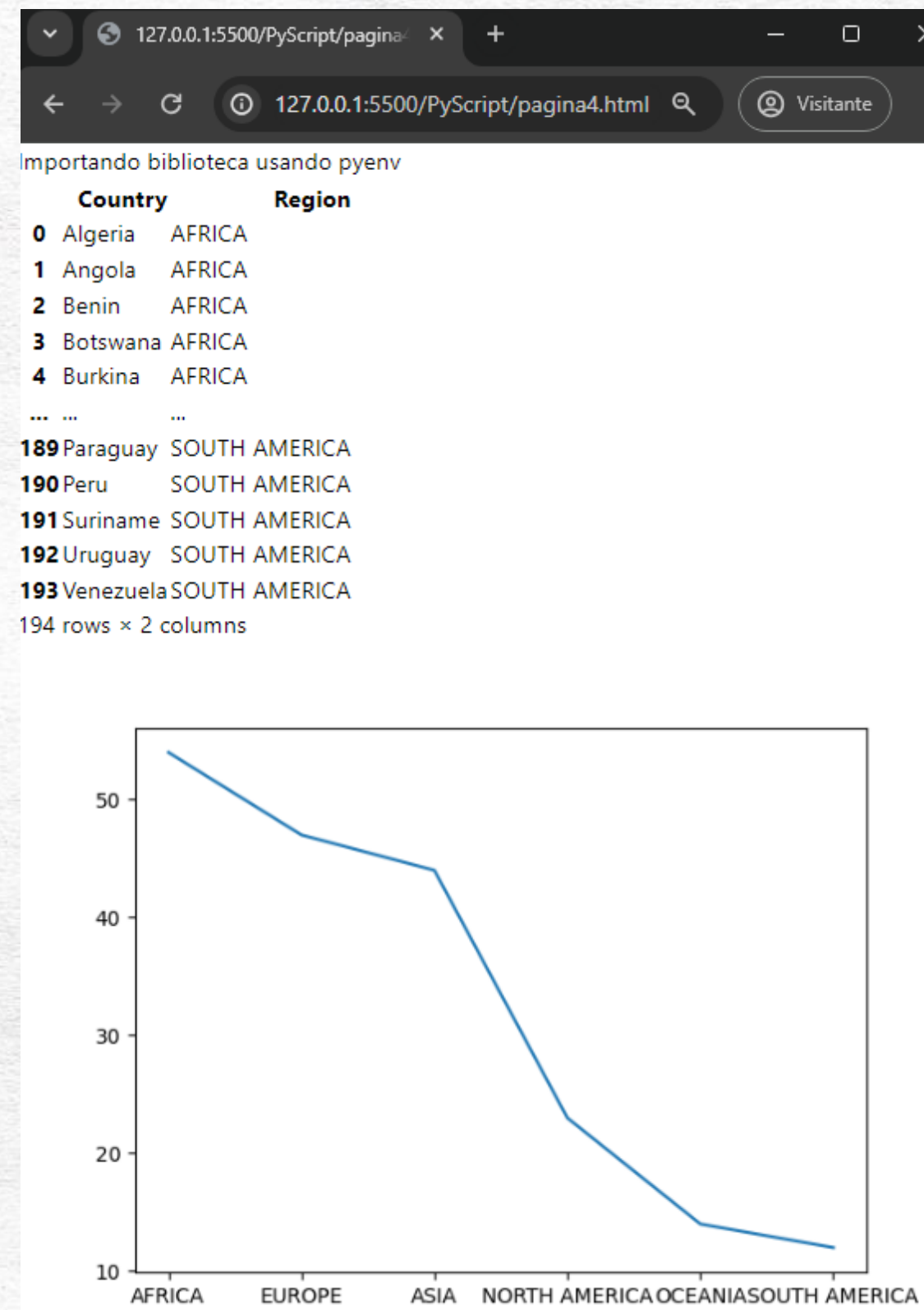
  </body>
</html>

<py-script>
  import pandas as pd
  import matplotlib.pyplot as plt

  from pyodide.http import open_url
  link = open_url("https://raw.githubusercontent.com/cs109/2014_data/master/countries.csv")
  tabela = pd.read_csv(link)
  pyscript.write("tabela_paises",tabela)

  contagem = tabela["Region"].value_counts()
  fig, eixo = plt.subplots()
  contagem.plot(ax=eixo)
  pyscript.write("grafico_qtde", fig)
</py-script>
</body>
</html>
```

Com isso, temos a **tabela** e o **gráfico** sendo exibidos na nossa página.



Agora, vamos para o nosso último exemplo: a capacidade de importar um arquivo Python externo para dentro do HTML utilizando o PyScript. Até o momento, vimos como escrever manualmente o código dentro das tags do PyScript, porém, podemos utilizar um arquivo **Python externo**.

Por exemplo, vamos pegar o código que utilizamos para criar a tabela e o gráfico do exemplo anterior e defini-lo dentro de um arquivo Python chamado **externo.py**.

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 from pyodide.http import open_url
5 link = open_url("https://raw.githubusercontent.com/cs109/2014_data/master/countries.csv")
6 tabela = pd.read_csv(link)
7 pyscript.write("tabela_paises", tabela)
8
9 contagem = tabela["Region"].value_counts()
10 fig, eixo = plt.subplots()
11 contagem.plot(ax=eixo)
12 pyscript.write("grafico_qtde", fig)
```

Na mesma pasta em que salvamos esse arquivo, podemos criar o nosso documento HTML chamado **pagina5.html**.



Dentro desse arquivo HTML, ao invés de utilizarmos a tag **<py-script>** para escrever todo o código Python como fizemos anteriormente, podemos apenas passar o atributo **src** com o caminho para o arquivo **externo.py**.

Para não precisar escrever todo o caminho, podemos informá-lo da seguinte forma: **./externo.py**. O **./** indicará para o PyScript que ele deve pegar o arquivo **externo.py** que está na mesma pasta do arquivo HTML.

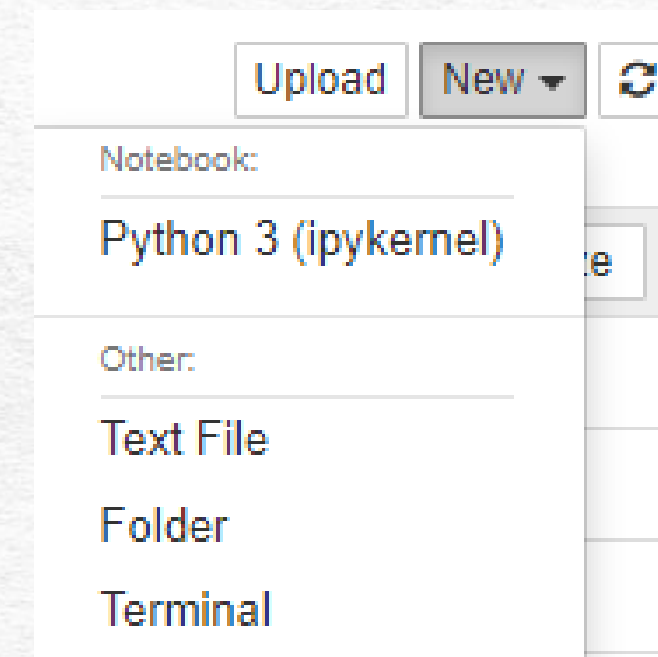
```
16
17     <py-script src="./externo.py"></py-script>
18     </body>
19 </html>
```

```
1 <html>
2   <head>
3     <link rel="stylesheet" href="https://pyscript.net/alpha/pyscript.css" />
4     <script defer src="https://pyscript.net/alpha/pyscript.js"></script>
5     <py-env>
6       - pandas
7       - matplotlib
8     </py-env>
9   </head>
10
11   <body>
12     <h1>Rodando scripts externos usando o PyScript</h1>
13
14     <div id="tabela_paises"></div>
15     <div id="grafico_qtde"></div>
16
17     <py-script src="./externo.py"></py-script>
18   </body>
19 </html>
```

Feito isso, o código funcionará corretamente, mas se você apenas tentar abrir o arquivo HTML no seu navegador, obterá um erro. Isso ocorre porque os navegadores não permitem importar arquivos de código local diretamente do computador.

Para fazer isso, ambos os arquivos precisam estar em um **servidor**. Para testar dessa forma, existem algumas possibilidades. Uma delas é utilizar o **VS Code** com a **extensão Live Server**.

Outra opção é criar um **servidor local** com o Jupyter Notebook. Para isso, clique em **New** na página inicial do Jupyter e selecione a opção **Terminal**.



Com o terminal aberto, navegue até a pasta onde estão os arquivos e execute o seguinte comando:

python -m http.server

```
O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PowerShell

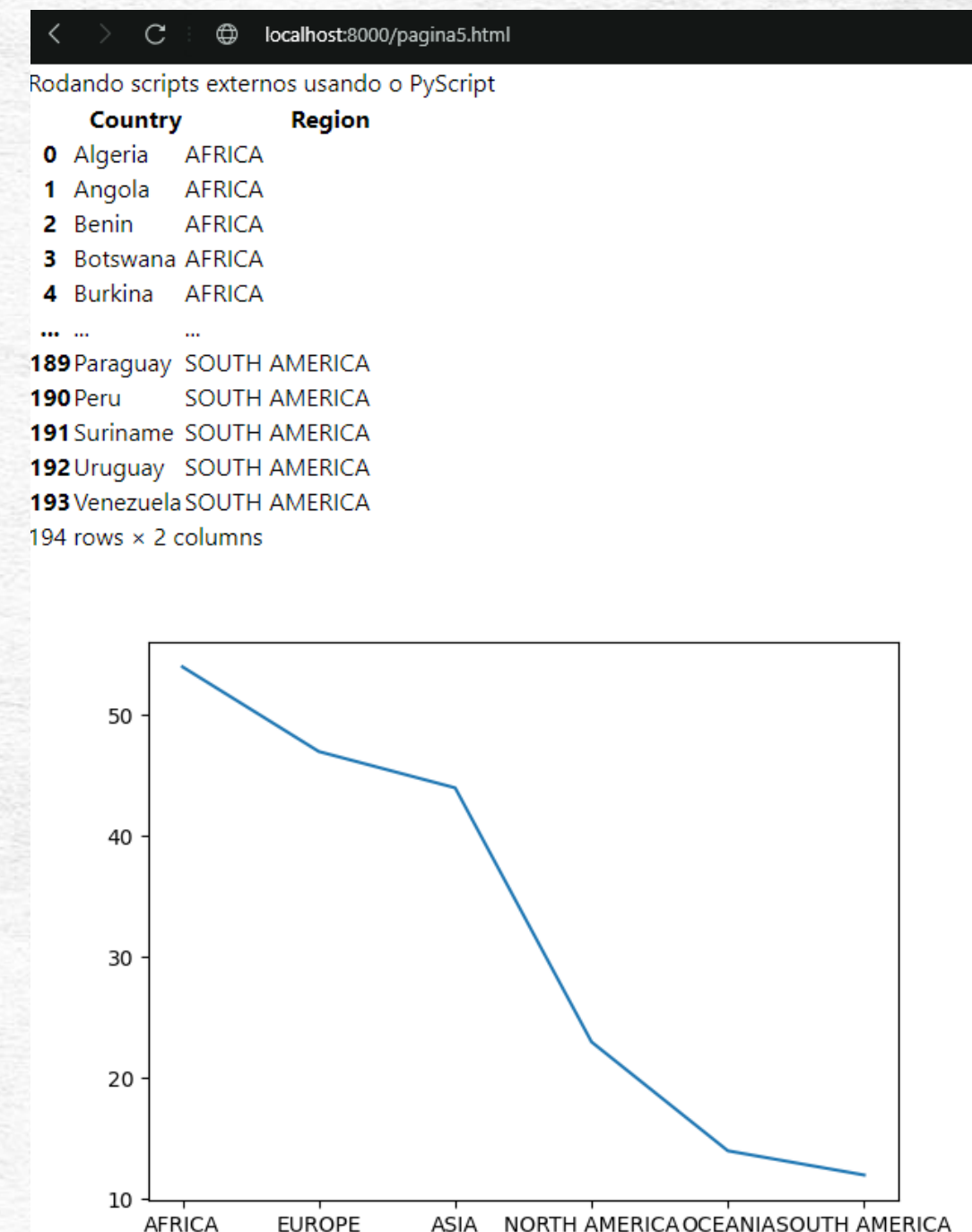
PS C:\Users\diego\OneDrive\Documentos> python -m http.server
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
```

Feito isso, acesse o endereço **http://localhost:8000/** no seu navegador. Localize o arquivo **pagina5.html** e clique sobre ele.

Directory listing for /

- [Adobe/](#)
- [Audacity/](#)
- [Camtasia/](#)
- [desktop.ini](#)
- [externo.py](#)
- [Gravações de som/](#)
- [pagina5.html](#)
- [Python Scripts/](#)
- [SQL Server Management Studio/](#)

Isso carregará a página HTML no seu navegador com os mesmos resultados obtidos anteriormente.





PYTHON

Módulo 54

INTRODUÇÃO A RECONHECIMENTO DE IMAGEM E FALA

Neste módulo você aprenderá como fazer reconhecimento de voz utilizando o Python. Para os exemplos dessa aula, eu utilizarei o **PyCharm**, mas você pode utilizar o editor de códigos ou a IDE de sua preferência.

SpeechRecognition:

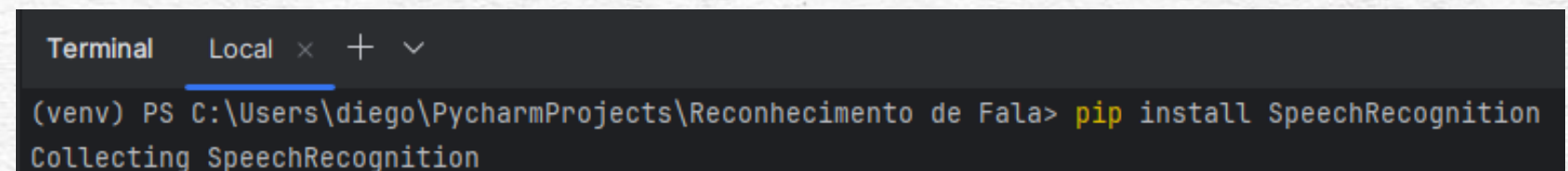
Para o reconhecimento de fala utilizaremos a biblioteca **SpeechRecognition** do Python. Essa é uma poderosa ferramenta que permite converter fala em texto usando diferentes motores de reconhecimento de fala.

A biblioteca **SpeechRecognition** permite o uso de diferentes **engines** e **APIs** de reconhecimento de fala como o Google Speech Recognition, Google Cloud Speech API, Sphinx, Wit.ai, Microsoft Azure Speech, Microsoft Bing Voice Recognition, IBM Speech to Text, Snowboy, OpenAI whisper, Houndify, entre outras opções.

Cada uma dessas opções tem suas próprias características, vantagens e limitações. A escolha de qual utilizar vai depender das necessidades do seu projeto.

No início, eu recomendo que você utilize as mais simples, em que você não precisa fazer nenhuma instalação ou configuração extra para utilizar, como a **Google Speech Recognition**, que usaremos nessa aula, ou a **Sphinx**.

Como de costume, nosso primeiro passo será instalar a biblioteca SpeechRecognition com o comando **pip install SpeechRecognition**.

A screenshot of a terminal window with a dark background. The title bar shows 'Terminal' and 'Local' with window control icons. The command prompt shows '(venv) PS C:\Users\diego\PycharmProjects\Reconhecimento de Fala> pip install SpeechRecognition'. The output shows 'Collecting SpeechRecognition'.

Além disso, precisamos instalar a biblioteca do **PyAudio** que nos permitirá capturar o áudio do microfone e utilizá-lo com os códigos em Python.

```
Terminal  Local x + v
(venv) PS C:\Users\diego\PycharmProjects\Reconhecimento de Fala> pip install pyaudio
Collecting pyaudio
```

Obs.: Em alguns casos pode ocorrer um erro na instalação do PyAudio. Caso isso ocorra, e seu sistema operacional seja o **Windows**, você pode fazer a instalação com o comando **pipwin install pyaudio**. Para outros sistemas operacionais é importante verificar no Google ou em sites como Stack Overflow qual o comando recomendado.

O PyAudio só é necessário para que possamos utilizar o **microfone** com o **SpeechRecognition**. Caso você queira apenas transcrever um áudio para texto você pode utilizar apenas a biblioteca SpeechRecognition.

Como Reconhecer Áudio do Microfone:

Feita a instalação das bibliotecas, vamos partir para o desenvolvimento do nosso primeiro exemplo.

Nele, iremos criar um objeto **recognizer** (“reconhecedor”), que fará a transcrição do áudio para texto, e esse áudio será obtido através do microfone do seu computador.

O primeiro passo será importar a biblioteca **SpeechRecognition** com o alias **sr** e criar o nosso **recognizer**.

```
1 import speech_recognition as sr
2
3 # - criar um "reconhecedor"
4 rec = sr.Recognizer()
```

Para o microfone, vamos criar uma instância da classe **Microphone** utilizando o **with**.

Essa construção garante que, ao sair da estrutura do **with**, os recursos associados ao microfone sejam automaticamente liberados, como parar a captura de áudio e fechar as conexões existentes.

```
1 import speech_recognition as sr
2
3 # - criar um "reconhecedor"
4 rec = sr.Recognizer()
5
6 # - ouvir o audio do microfone
7 with sr.Microphone() as microphone:
```

Caso você tenha mais de um microfone conectado ao seu computador, pode executar o código abaixo para identificar uma lista com os microfones presentes:

```
print(sr.Microphone().list_microphone_names())
```

```
1 import speech_recognition as sr
2
3 print(sr.Microphone().list_microphone_names())
```

```
['Mapeador de som da Microsoft - Input', 'Microphone (High Definition Aud',
```

Dentro dessa lista, basta identificar o microfone que você utiliza em seu computador e qual o índice dele. Lembrando que as listas em Python começam no **índice 0**.

Feito isso, você irá informar o parâmetro **device_index** para a classe **Microphone** com o índice correspondente ao seu microfone.

```
1 import speech_recognition as sr
2
3 # - criar um "reconhecedor"
4 rec = sr.Recognizer()
5
6 # - ouvir o audio do microfone
7 with sr.Microphone(device_index=3) as microphone:
```

Caso você queira utilizar o microfone padrão, basta deixar a classe sem esse parâmetro.

Para diminuir o ruído do ambiente captado pelo microfone, utilizaremos o método **adjust_for_ambient_noise**, passando para ele o objeto **microfone** que acabamos de criar.

```
1 import speech_recognition as sr
2
3 # - criar um "reconhecedor"
4 rec = sr.Recognizer()
5
6 # - ouvir o audio do microfone
7 with sr.Microphone(device_index=3) as microfone:
8     rec.adjust_for_ambient_noise(microfone)
```

Esse método capta o áudio do microfone por um breve período e calcula o nível de ruído de fundo. Ele ajusta automaticamente os parâmetros de reconhecimento de fala para distinguir melhor a voz do ruído ambiente.

Assim, a fala, que geralmente é muito mais alta do que o ruído de fundo, pode ser identificada com maior precisão.

Como inicialmente o programa começa reconhecendo e tratando o ruído ambiente, vamos adicionar uma mensagem após esse processo informando ao usuário que ele pode começar a falar.

Assim, saberemos exatamente o momento em que o programa já terminou de capturar e identificar o ruído de fundo e está pronto para captar a nossa fala.

O método que utilizaremos para captar a nossa fala é o método **listen**.

```
1 import speech_recognition as sr
2
3 # - criar um "reconhecedor"
4 rec = sr.Recognizer()
5
6 # - ouvir o audio do microfone
7 with sr.Microphone(device_index=3) as microfone:
8     rec.adjust_for_ambient_noise(microfone)
9
10     print("Pode começar a falar:")
11     audio = rec.listen(microfone)
```

Em seguida, vamos pegar esse áudio e transcrevê-lo. Para isso, podemos criar uma estrutura de **try** e **except** para tratar possíveis erros em que o áudio não seja captado.

Ou seja, se o áudio for captado com sucesso, o código irá transcrevê-lo, caso contrário, exibirá uma mensagem de erro para o usuário.

Para transcrever o áudio, utilizaremos o método **recognize_google**, que utiliza a **engine do Google** e requer apenas o áudio capturado e o idioma em que o áudio está para fazer a transcrição.

```
1 import speech_recognition as sr
2
3 # - criar um "reconhecedor"
4 rec = sr.Recognizer()
5
6 # - ouvir o audio do microfone
7 with sr.Microphone(device_index=3) as microfone:
8     rec.adjust_for_ambient_noise(microfone)
9
10    print("Pode começar a falar:")
11    audio = rec.listen(microfone)
12
13    try:
14        # reconhece o audio e traduz pra texto
15        texto = rec.recognize_google(audio, language="pt-BR")
16        print(texto)
17    except:
18        print("Não peguei áudio nenhum")
```

Um ponto interessante é que, por padrão, esse método interromperá a captura de voz após 0.8 segundos de silêncio. Caso você queira alterar esse tempo para mais ou para menos, deverá utilizar o método `pause_threshold`.

```
1 import speech_recognition as sr
2
3 # - criar um "reconhecedor"
4 rec = sr.Recognizer()
5
6 # - ouvir o audio do microfone
7 with sr.Microphone() as microfone:
8     rec.adjust_for_ambient_noise(microfone)
9
10    print("Pode começar a falar:")
11    rec.pause_threshold = 1
12    audio = rec.listen(microfone)
13
14    try:
15        # reconhece o audio e traduz pra texto
16        texto = rec.recognize_google(audio, language="pt-BR")
17        print(texto)
18    except:
19        print("Não peguei áudio nenhum")
```

Dessa forma, o código aguardará **1 segundo** de silêncio para interromper a gravação.

Agora, vamos modificar o exemplo anterior para realizar a gravação do áudio capturado em um arquivo. Em vez de transcrever diretamente o que falamos no microfone, esse código irá salvar o áudio em um **arquivo WAV**.

Para isso, utilizaremos a estrutura **with**, que garante que o arquivo seja fechado corretamente após a escrita. Usaremos a função **open** para criar um arquivo em Python, passando o nome do arquivo com a extensão e o modo de abertura "**wb**", que indica escrita binária.

Dentro dessa estrutura, utilizaremos o método **write**, que escreverá os dados do áudio no arquivo **audio.wav**. Para obter esses dados, usaremos o método **get_wav_data()**, que converte o áudio capturado pelo microfone em um formato WAV.

```
1 import speech_recognition as sr
2
3 rec = sr.Recognizer()
4
5 with sr.Microphone(device_index=3) as microfone:
6     rec.adjust_for_ambient_noise(microfone)
7     print("Pode começar a falar")
8     audio = rec.listen(microfone)
9
10 # salvar o audio
11 with open("audio.wav", "wb") as arquivo:
12     arquivo.write(audio.get_wav_data())
```

A biblioteca SpeechRecognition permite ler áudios em quatro formatos diferentes: **raw**, **wav**, **aiff** e **flac**.

Feito isso, pegaremos o arquivo de áudio que acabamos de criar e utilizaremos a biblioteca **SpeechRecognition** para transcrever o seu conteúdo em texto.

Primeiro, importamos a biblioteca e criamos o **recognizer**, assim como fizemos nos exemplos anteriores.

Em seguida, abrimos o arquivo de áudio que desejamos transcrever utilizando a classe **AudioFile** dentro de uma estrutura **with**. Essa classe será responsável por ler o arquivo de áudio.

Feito isso, usamos o método **record** do objeto **recognizer** para armazenar o conteúdo desse áudio em uma variável, que passaremos para o método **recognize_google** para transcrever o áudio como texto.

```
1 import speech_recognition as sr
2
3 rec = sr.Recognizer()
4
5 with sr.AudioFile("audio.wav") as arquivo_audio:
6     audio = rec.record(arquivo_audio)
7     texto = rec.recognize_google(audio, language="pt-BR")
8     print(texto)
```

Continuando com o reconhecimento de áudio, veremos agora como fazer a transcrição de áudio em tempo real, utilizando múltiplas threads.

Para isso, usaremos a biblioteca **SpeechRecognition** para capturar áudio continuamente do microfone e transcrever o conteúdo em tempo real, até que um comando para parar seja detectado.

Primeiro, importamos as bibliotecas necessárias (**speech_recognition** e **time**) e criamos uma função chamada **tratar_audio**, que lidará com a transcrição do áudio capturado.

Dentro dessa função, utilizamos o método **recognize_google** para transcrever o áudio e verificamos se o comando "**encerrar gravação**" está presente no texto transcrito.

Se esse comando for detectado, definimos a **variável global acabou** como **True** para sinalizar que devemos encerrar a captura de áudio.

Como queremos modificar o valor de uma variável global dentro de uma função, precisamos declarar que essa variável é global.

Para isso, usamos a palavra-chave **global** antes do nome da variável no início da função.

```
1 import speech_recognition as sr
2 import time
3
4 def tratar_audio(rec, audio):
5     global acabou
6     # tratar o meu audio
7     try:
8         texto = rec.recognize_google(audio, language="pt-BR")
9         print(texto)
10        if "encerrar gravação" in texto:
11            acabou = True
12    except:
13        print("Não escutei")
```

Em seguida, inicializamos a variável acabou como **False** e criamos o objeto **recognizer**.

```
1 import speech_recognition as sr
2 import time
3
4 def tratar_audio(rec, audio):
5     global acabou
6     # tratar o meu audio
7     try:
8         texto = rec.recognize_google(audio, language="pt-BR")
9         print(texto)
10        if "encerrar gravação" in texto:
11            acabou = True
12    except:
13        print("Não escutei")
14
15 acabou = False
16
17 rec = sr.Recognizer()
```

Configuramos o microfone para capturar o áudio, ajustando os níveis de ruído ambiente e definindo o tempo de pausa (`pause_threshold`) para 0.8 segundos dentro do bloco `with`.

```
19 with sr.Microphone(device_index=3) as microfone:
20     rec.adjust_for_ambient_noise(microfone)
21     rec.pause_threshold = 0.8
22     print("Pode começar a falar:")
```

Para capturar o áudio em tempo real, utilizamos o método `listen_in_background`, que inicia a captura de áudio em uma `thread` separada. Passamos a função `tratar_audio` para processar o áudio capturado.

Dessa forma, o programa continua a ser executado sem esperar pela conclusão do reconhecimento de áudio.

Esse método retorna uma função que pode ser chamada para parar a escuta em segundo plano, armazenada na variável `parar_ouvir`.

```
24 # thread 1
25 parar_ouvir = rec.listen_in_background(microfone, tratar_audio)
```

Enquanto isso, em outra `thread`, utilizamos um loop `while` para manter o programa em execução até que a variável `acabou` seja definida como `True`.

```
27 # thread 2
28 while True:
29     time.sleep(0.1)
30     if acabou == True: # vou falar pra ele encerrar a gravação
31         break
```

Por fim, paramos a captura de áudio chamando o método retornado por `listen_in_background`.

Para isso, chamamos a função `parar_ouvir` passando o parâmetro `wait_for_stop=False`, para garantir que o método não espere até que o processo de escuta atual termine.

```
33 # thread 1
34 parar_ouvir(wait_for_stop=False)
```

Isso garante que o programa principal possa encerrar rapidamente a captura de áudio quando o comando "encerrar gravação" for detectado.

Agora veremos como fazer o reconhecimento de imagens usando Python e a biblioteca **OpenCV**.

Para os exemplos desta aula, utilizarei o **Jupyter Notebook**, mas você pode usar o editor de código ou a IDE de sua preferência.

OpenCV:

A **OpenCV** (Open Source Computer Vision Library) é uma biblioteca de código aberto amplamente utilizada para processamento de imagens, reconhecimento de objetos, análise de vídeos e muito mais.

Essa biblioteca é capaz de integrar e controlar a sua webcam, além de permitir visualizar e trabalhar com a tela do seu computador como se ela fosse uma **lista Python**, em que cada pixel é um elemento dessa lista.

Cada pixel segue o padrão **BGR** (Blue, Green e Red ou Azul, Verde e Vermelho), variando de **0** a **255**. Esses valores indicam a quantidade de azul, verde e vermelho presente naquele pixel e formam a cor dele.

Quando a OpenCV está trabalhando com uma imagem no seu computador, ela considera cada pixel presente na tela e o valor de BGR contido nele.

MediaPipe:

Além da OpenCV, vamos utilizar a biblioteca **MediaPipe**, um framework de código aberto desenvolvido pelo **Google** que fornece ferramentas para a criação de aplicações de processamento de mídia.

Para começar, nosso primeiro passo será instalar as bibliotecas necessárias: **OpenCV** e **MediaPipe**. Para isso, abra o terminal do seu editor de código e execute os comandos:

```
pip install opencv  
pip install mediapipe
```

```
(base) C:\Users\Python>pip install opencv-python
```

```
(base) C:\Users\Python>pip install mediapipe
```

Feita a instalação das bibliotecas, vamos partir para o desenvolvimento do nosso projeto. Nele, iremos reconhecer e vincular a webcam do computador ao Python e, em seguida, reconhecer as mãos do usuário.

Criando a Conexão do Python com a Webcam:

O primeiro passo será importar a biblioteca **OpenCV** como **cv2** e a **MediaPipe** com o alias de **mp**.

```
1 import cv2
2 import mediapipe as mp
```

Em seguida, vamos configurar e detectar a webcam para ser usada no Python utilizando a biblioteca OpenCV. Para isso, criaremos um objeto **webcam** utilizando a função **VideoCapture()**.

Essa função recebe como argumento o **índice da webcam** que você deseja utilizar, caso tenha mais de uma webcam conectada ao seu computador. O **índice 0** reconhece a webcam padrão do seu sistema.

```
1 import cv2
2 import mediapipe as mp
3
4 # vincular a webcam ao python
5 webcam = cv2.VideoCapture(0) # cria a conexão com a webcam
```

Obs.: É importante notar que dois programas não conseguem utilizar a mesma webcam simultaneamente. Portanto, você não conseguirá executar esse código corretamente caso já esteja utilizando a webcam do seu computador no momento.

Com isso, conseguimos estabelecer a conexão da nossa webcam com o código. No entanto, para ler as informações capturadas por ela, precisamos utilizar o método **read()**.

Esse método captura e lê um frame da webcam e retorna dois valores.

O primeiro é um **valor booleano** que indica se a captura do frame foi bem-sucedida ou não. Se o valor for **True**, a captura funcionou; se for **False**, ocorreu algum problema ao tentar capturar e ler o frame.



O segundo é o **frame capturado** em si, que é uma imagem representada como uma **matriz do NumPy**. Essa matriz contém os **valores dos pixels** da imagem em formato BGR (Blue, Green, Red).

```
(True, array([[ 88, 110, 106],
              [ 90, 110, 107],
              [ 92, 111, 109],
              ...,
              [106, 114, 109],
              [106, 113, 109],
              [107, 114, 111]]),
```

Porém, não queremos capturar apenas um frame da nossa webcam, mas sim capturar o vídeo em tempo real, ou seja, ela precisará capturar e ler diversos frames por segundo.

Para isso, primeiro vamos utilizar o método **isOpened()** para verificar se a webcam está aberta corretamente.

Se estiver aberta, capturamos o primeiro frame com o método **read()**. Vamos armazenar os valores retornados nas variáveis **validacao** e **frame**.

Como vimos, **validacao** será uma variável booleana, sendo **True** caso o frame tenha sido capturado e **False**, caso contrário.

```
1 import cv2
2 import mediapipe as mp
3
4 # vincular a webcam ao python
5 webcam = cv2.VideoCapture(0) # cria a conexão com a webcam
6
7 # Verificar a conexão
8 if webcam.isOpened():
9     # ler a webcam
10     validacao, frame = webcam.read()
```

Em seguida, podemos utilizar um loop **while** para capturarmos diversos frames. Enquanto a **validacao** for verdadeira (True), continuamos capturando os frames da webcam.

Dentro desse loop, podemos usar a função **imshow** para exibir a imagem que está sendo capturada pela webcam.

Para essa função, passamos dois argumentos. O nome da janela (**window_name**) que exibirá a imagem e pode ser usado para referenciar a janela em outras funções do OpenCV. E também a imagem (**frame**) que desejamos exibir, o frame que estamos capturando.

```
1 import cv2
2 import mediapipe as mp
3
4 # vincular a webcam ao python
5 webcam = cv2.VideoCapture(0) # cria a conexão com a webcam
6
7 # Verificar a conexão
8 if webcam.isOpened():
9     # Ler a webcam
10    validacao, frame = webcam.read()
11    # Loop infinito - pegar vários frames
12    while validacao:
13        # pegar o próximo frame da tela
14        validacao, frame = webcam.read()
15
16        # mostrar o frame da webcam que o python ta vendo
17        cv2.imshow("Video da Webcam", frame)
```

Além disso, vamos utilizar a função **waitKey** para fazer com que o programa espere por **2 milissegundos** entre cada captura de frame. Com isso, teremos uma captura de aproximadamente **30 fps** (frames por segundo).

Por fim, além de utilizarmos a função **waitKey** para que o programa espere, podemos utilizar essa mesma função para armazenar uma tecla.

Com isso, conseguimos verificar se a tecla **ESC** (código ASCII 27) foi pressionada e, caso tenha sido, o loop será interrompido, parando a captura do vídeo.

```
1 import cv2
2 import mediapipe as mp
3
4 # vincular a webcam ao python
5 webcam = cv2.VideoCapture(0) # cria a conexão com a webcam
6
7 # Verificar a conexão
8 if webcam.isOpened():
9     # Ler a webcam
10    validacao, frame = webcam.read()
11    # Loop infinito - pegar vários frames
12    while validacao:
13        # pegar o próximo frame da tela
14        validacao, frame = webcam.read()
15
16        # mostrar o frame da webcam que o python ta vendo
17        cv2.imshow("Video da Webcam", frame)
18        # definir espera do Python
19        tecla = cv2.waitKey(2)
20        # mandar ele parar o código se eu clicar no ESC
21        if tecla == 27:
22            break
```

Após sair do loop, liberamos a webcam com `webcam.release()` e fechamos todas as janelas abertas pelo OpenCV com `cv2.destroyAllWindows()`.

```
1 import cv2
2 import mediapipe as mp
3
4 # vincular a webcam ao python
5 webcam = cv2.VideoCapture(0) # cria a conexão com a webcam
6
7 # Verificar a conexão
8 if webcam.isOpened():
9     # ler a webcam
10    validacao, frame = webcam.read()
11    # loop infinito - pegar vários frames
12    while validacao:
13        # pegar o próximo frame da tela
14        validacao, frame = webcam.read()
15
16        # mostrar o frame da webcam que o python ta vendo
17        cv2.imshow("Video da Webcam", frame)
18        # definir espera do Python
19        tecla = cv2.waitKey(2)
20        # mandar ele parar o código se eu clicar no ESC
21        if tecla == 27:
22            break
23
24 # Encerrar a conexão
25 webcam.release()
26 cv2.destroyAllWindows()
```

Executando esse código, o programa capturará a imagem da sua webcam e ficará exibindo-a, até que você pressione a tecla ESC.

Detecção das Mãos:

Após configurarmos a conexão com a webcam e começarmos a capturar os frames, precisamos inicializar o **MediaPipe** para detectar e reconhecer as mãos do usuário.

Esse framework nos permite aplicar rapidamente técnicas de **inteligência artificial** ao nosso programa. Ou seja, podemos utilizar uma IA treinada pelo Google para o reconhecimento de mãos diretamente no nosso código.

Para utilizá-la, além de importar a biblioteca, é necessário configurarmos os módulos responsáveis pelo reconhecimento de mãos. Dentro do MediaPipe, esses módulos são chamados de soluções (**solutions**).

Você pode visualizar a documentação completa a respeito do MediaPipe na página da documentação oficial, [clcando aqui](#).

Como queremos detectar as mãos do usuário, precisamos configurar o módulo `mp.solutions.hands`, que contém todas as funcionalidades necessárias para o reconhecimento de mãos.



Também utilizaremos o módulo `mp.solutions.drawing_utils`, que fornece os utilitários para desenhar as marcações das mãos detectadas sobre os frames capturados.

Feito isso, podemos inicializar o reconhecimento das mãos criando uma instância da classe **Hands** do módulo `mp.solutions.hands`.

Essa classe será responsável por configurar o processo de detecção, aplicando os modelos de aprendizado da IA para identificar e rastrear as mãos nos frames capturados.

```
1  import cv2
2  import mediapipe as mp
3
4  # vincular a webcam ao python
5  webcam = cv2.VideoCapture(0) # cria a conexão com a webcam
6
7  # inicializando o mediapipe
8  reconhecimento_maos = mp.solutions.hands
9  desenho_mp = mp.solutions.drawing_utils
10 maos = reconhecimento_maos.Hands()
```

Além de configurarmos o MediaPipe, precisamos converter as cores dos frames capturados. Isso é necessário porque, como já mencionado, o **OpenCV** utiliza o padrão **BGR** para cores, enquanto o **MediaPipe** usa o padrão **RGB**.

Para isso, utilizaremos a função do OpenCV chamada **cvtColor**. Essa função recebe como argumento a imagem de origem (frame capturado pela webcam em BGR) e, como segundo argumento, o código de conversão de cores desejado.

Nesse caso, como queremos converter de BGR para RGB, usamos o código **cv2.COLOR_BGR2RGB**.

Vamos aplicar essa função dentro do nosso loop infinito para tratarmos todos os frames capturados.

```
1  import cv2
2  import mediapipe as mp
3
4  # vincular a webcam ao python
5  webcam = cv2.VideoCapture(0) # cria a conexão com a webcam
6
7  # inicializando o mediapipe
8  reconhecimento_maos = mp.solutions.hands
9  desenho_mp = mp.solutions.drawing_utils
10 maos = reconhecimento_maos.Hands()
11
12
13 # Verificar a conexão
14 if webcam.isOpened():
15     # Ler a webcam
16     validacao, frame = webcam.read()
17     # loop infinito - pegar vários frames
18     while validacao:
19         # pegar o próximo frame da tela
20         validacao, frame = webcam.read()
21
22         #converte BGR em RGB
23         frameRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

Por fim, vamos adicionar o processamento dos frames e detecção das mãos utilizando a função **process** da instância **maos**.

Essa é uma função do **MediaPipe** que processa a imagem fornecida, no caso nossos frames convertidos para o padrão RGB (**frameRGB**), e detecta a presença de mãos neles.

Como resposta, essa função retorna um objeto que contém informações sobre as mãos detectadas na imagem. Por padrão, o MediaPipe detecta até **duas mãos** por imagem.

Depois de processar o frame capturado pela webcam, vamos verificar se alguma mão foi detectada verificando a propriedade **multi_hand_landmarks** do objeto retornado.

Essa é uma **lista** de objetos **HandLandmark** que representam as mãos na imagem. Se esta lista não estiver vazia, significa que pelo menos uma mão foi reconhecida.

Com as mãos detectadas, podemos iterar sobre cada mão presente na lista **multi_hand_landmarks** para extrair e desenhar as marcações presentes.

```
1 import cv2
2 import mediapipe as mp
3
4 # vincular a webcam ao python
5 webcam = cv2.VideoCapture(0) # cria a conexão com a webcam
6
7 # inicializando o mediapipe
8 reconhecimento_maos = mp.solutions.hands
9 desenho_mp = mp.solutions.drawing_utils
10 maos = reconhecimento_maos.Hands()
11
12
13 # Verificar a conexão
14 if webcam.isOpened():
15     # ler a webcam
16     validacao, frame = webcam.read()
17     # loop infinito - pegar vários frames
18     while validacao:
19         # pegar o próximo frame da tela
20         validacao, frame = webcam.read()
21
22         #converte BGR em RGB
23         frameRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
24
25         # desenhar a mão
26         lista_maos = maos.process(frameRGB)
27         if lista_maos.multi_hand_landmarks:
28             for mao in lista_maos.multi_hand_landmarks:
```

Essas marcações são chamadas de **landmark**, pontos de referência que extraímos para cada mão detectada. Cada ponto de referência representa uma parte específica da mão (dedos, articulações, etc.) contida em uma lista de pontos de referência (**landmarks**).

Finalmente, podemos desenhar essas marcações detectadas sobre o frame original utilizando a função **draw_landmarks**.

Essa função do MediaPipe desenha os pontos de referência e as conexões entre eles sobre cada frame capturado pela webcam.

Para essa função, precisamos passar a imagem (**frame**), a lista de pontos de referência detectados (a mão detectada, **mao**) e as conexões entre os pontos de referência que devem ser desenhadas (**HAND_CONNECTIONS**).

A propriedade **HAND_CONNECTIONS** define as conexões entre os pontos de referência, sendo formada por uma lista de tuplas, onde cada tupla contém dois índices que representam os pontos conectados.

```
1 import cv2
2 import mediapipe as mp
3
4 # vincular a webcam ao python
5 webcam = cv2.VideoCapture(0) # cria a conexão com a webcam
6
7 # inicializando o mediapipe
8 reconhecimento_maos = mp.solutions.hands
9 desenho_mp = mp.solutions.drawing_utils
10 maos = reconhecimento_maos.Hands()
11
12
13 # Verificar a conexão
14 if webcam.isOpened():
15     # Ler a webcam
16     validacao, frame = webcam.read()
17     # Loop infinito - pegar vários frames
18     while validacao:
19         # pegar o próximo frame da tela
20         validacao, frame = webcam.read()
21
22         #converte BGR em RGB
23         frameRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
24
25         # desenhar a mão
26         lista_maos = maos.process(frameRGB)
27         if lista_maos.multi_hand_landmarks:
28             for mao in lista_maos.multi_hand_landmarks:
29                 print(mao.landmark)
30                 desenho_mp.draw_landmarks(frame, mao, reconhecimento_maos.HAND_CONNECTIONS)
31
32         # mostrar o frame da webcam que o python ta vendo
33         cv2.imshow("Video da Webcam", frame)
34         # definir espera do Python
35         tecla = cv2.waitKey(2)
36         # mandar ele parar o código se eu clicar no ESC
37         if tecla == 27:
38             break
39
40 # Encerrar a conexão
41 webcam.release()
42 cv2.destroyAllWindows()
```

Para visualizar as coordenadas desses pontos de referência podemos printar o `mao.landmark`.

Executando esse código, o nosso programa estará completo para capturar a imagem da sua webcam e ficar exibindo-a com as marcações das mãos detectadas até que você pressione a tecla ESC.

```
[x: 0.1437034010887146  
y: 0.7913005352020264  
z: -4.2965250031556934e-05  
, x: 0.1911599338054657  
y: 0.7711524367332458  
z: -0.050874460488557816  
, x: 0.21824640035629272  
y: 0.7327916026115417  
z: -0.081874318420887
```



Para finalizar este módulo, vamos explorar e entender como funciona um projeto de ciência de dados que tem por objetivo treinar uma inteligência artificial para o reconhecimento de imagem e a solução de **CAPTCHAs**.

Todo projeto que envolve reconhecimento de imagem e extração de informações de uma imagem lida necessariamente com Inteligência Artificial.

Uma imagem é composta por pequenos quadrados, cada um com uma cor específica, chamados de **pixels**.

Portanto, para extrair informações de uma imagem, é preciso desenvolver um programa que consiga analisar esses pixels, identificar padrões nas cores e transformar isso em dados.

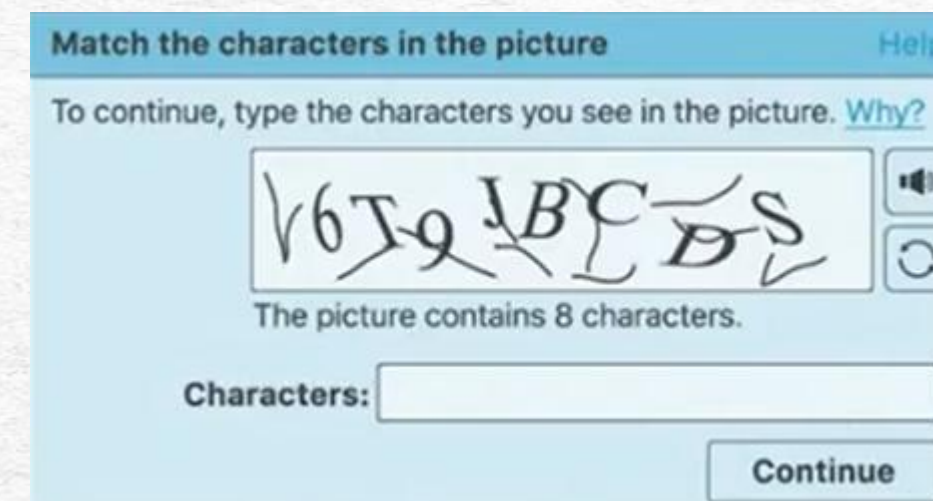
Por exemplo, para reconhecer o texto de um CAPTCHA, esse padrão de cores precisa ser interpretado como letras que formam uma palavra.

Esta aula é para que você entenda a lógica desse procedimento, quais etapas precisamos pensar e seguir para solucionar esse desafio.

CAPTCHA:

O CAPTCHA é uma ferramenta criada com o objetivo de impedir que códigos, bots e automações acessem determinados sites e serviços.

Os tipos mais comuns de CAPTCHA são os de **texto**, em que precisamos digitar letras e números, e os CAPTCHAs de **imagem**, em que precisamos selecionar partes da imagem onde aparece um determinado objeto.



Ciência de Dados – Processo de Análise

A primeira etapa para solucionar nosso desafio é buscar **informações sobre o tipo de CAPTCHA** que precisa ser resolvido e em que momento, dentro do processo que estamos automatizando, é necessário passar por ele.

Por exemplo, no caso da emissão de notas fiscais da Hashtag pelo site da Nota Carioca, assim que acessamos o site, no momento de fazer o login, é preciso resolver um CAPTCHA do tipo texto.

O próximo passo é analisar qual a forma mais fácil de passar por esse CAPTCHA. Verificar se existe alguma **API** ou outra forma simplificada de superá-lo, antes de partir para a criação de uma inteligência artificial.

Em alguns casos, é mais simples inserir o CAPTCHA manualmente durante o processo da sua automação, quando esse CAPTCHA só é exigido uma única vez, como no exemplo das notas fiscais, do que criar toda uma IA que consiga passar por ele.

Ler uma Imagem:

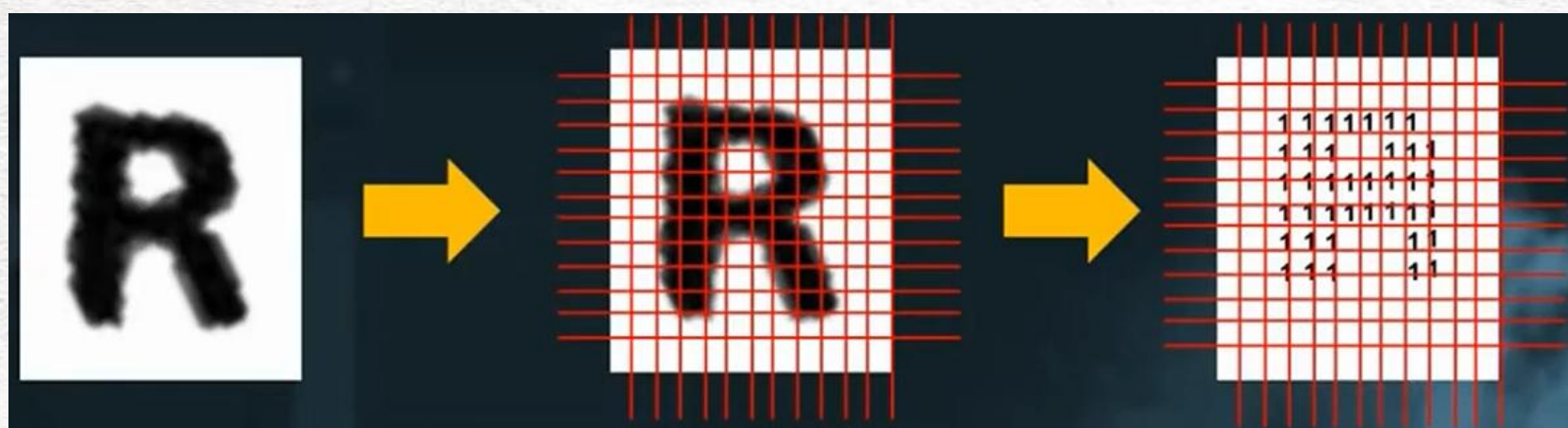
A primeira etapa da resolução do nosso problema é entender como o computador consegue ler uma imagem.

Em um CAPTCHA, podemos ter, por exemplo, a imagem da letra "R". Essa imagem é composta por diversos pixels, em que cada pixel é a menor unidade dela que pode ser manipulada pelo computador.

Cada um desses pequenos quadrados tem um valor atribuído a ele que representa a cor ou a intensidade de luz naquele ponto específico.

No caso de uma imagem preta e branca, o computador pode atribuir o valor **1** para os pixels da cor **preta** e **0** para os pixels em **branco**.

Dessa forma, os pixels que fazem parte da letra "R" são codificados como 1, e os pixels que fazem parte do fundo são codificados como 0.



Em imagens coloridas, esses pixels podem ser identificados de acordo com o código RGB ou hexadecimal. Já em imagens em tons de cinza, podemos utilizar a intensidade luminosa. Isso dependerá da ferramenta utilizada para ler e identificar a imagem.

Passo a Passo Solução:

Entendida a forma como o computador interpreta uma imagem, precisamos definir o passo a passo para a solução desse desafio. Para isso, precisamos seguir cinco passos:

1. Pegar a imagem do CAPTCHA
2. Limpar ruídos das imagens
3. Separar o CAPTCHA em letras
4. Identificar cada letra
5. Preencher o CAPTCHA

Passo 1 – Pegar a Imagem do CAPTCHA

Para pegar a imagem do CAPTCHA que queremos solucionar, podemos utilizar algumas bibliotecas do Python capazes de selecionar e salvar a imagem, ou então tirar um print da tela.

Podemos utilizar bibliotecas de automação como o **Selenium** ou o **PyAutoGUI** para esse tipo de tarefa.

Passo 2 – Limpar ruídos das imagens

Após extrair e salvar a imagem do CAPTCHA, precisamos **limpar os ruídos** presentes nela para que a letra fique mais clara. Para isso, podemos utilizar a biblioteca de processamento de imagens **OpenCV**.

Essa biblioteca tem funcionalidades para limpar os ruídos presentes em uma imagem.

Ela basicamente analisa a imagem, identifica a escala de cores presentes nela e consegue separar o que é ruído do que é imagem em si.

Por exemplo, considerando que **0** seja totalmente branco e **1** seja totalmente preto.

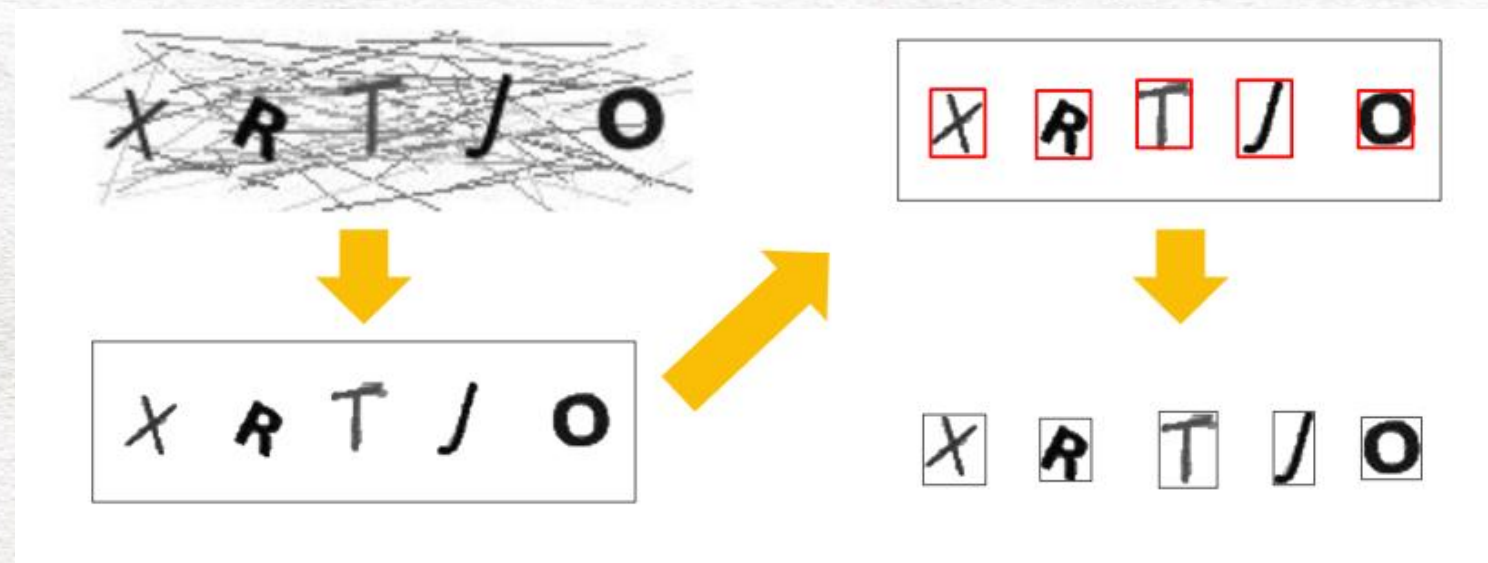
A biblioteca define um valor, por exemplo, **0,7**; valores a partir de 0,7 serão considerados como **preto** (1), e o restante será tratado como **branco** (0).



Com os valores preenchidos em 0 e 1, o Python conseguirá trabalhar melhor com o reconhecimento da imagem.

Passo 3 – Separar o CAPTCHA em letras

Ainda utilizando a biblioteca **OpenCV**, podemos utilizar as funções de contorno presentes nela. Essa funcionalidade reconhece o contorno das letras e consegue separá-las em arquivos de imagens distintas.



Passo 4 – Identificar cada Letra

Nessa etapa, vamos precisar utilizar uma IA que identifique qual é a letra de acordo com a imagem. Podemos buscar por uma IA pronta que faça esse processo ou criar uma a partir do zero.

O processo de desenvolver e treinar uma inteligência artificial envolve pegar milhares de cópias do CAPTCHA, fazer o processo de limpeza e separação das letras como vimos, para então desenvolver o aprendizado supervisionado da IA, onde iremos rotular cada letra e passar os dados rotulados para uma rede neural.

Esse processo de rotulagem das letras pode ser feito manualmente, o que dá bastante trabalho, em que pegamos cada letra e colocamos em uma pasta correspondente ao rótulo dela.

Alternativamente, podemos utilizar um método semi-automático, fazendo alguns exemplos manualmente, passando esses exemplos para a biblioteca OpenCV e utilizando-a para comparar automaticamente as demais letras obtidas com os rótulos criados manualmente.

Feito isso, podemos revisar e corrigir os rótulos que deram errado. Esse processo semi-automático irá poupar nosso trabalho, mas terá uma chance de acerto menor do que fazer tudo manualmente.

Terminada essa etapa, teremos dados suficientes para criar e treinar nossa inteligência artificial, passando os dados rotulados para uma rede neural como o **Keras** ou **TensorFlow**.

Uma rede neural é um modelo de IA muito utilizado para reconhecer padrões, como esse que queremos alcançar com nosso projeto.

A estrutura dessa rede é inspirada no funcionamento do cérebro humano, então é como se tivéssemos “neurônios” que recebem uma informação, realizam cálculos, aplicam funções a essa informação e produzem uma saída que é enviada para outro neurônio.

Essa rede é dividida em camadas de entrada, que recebem os dados iniciais, camadas ocultas que processam as entradas por meio de múltiplos neurônios, e a camada de saída que produz o resultado final da rede.

Nesse caso, a rede irá fragmentar a imagem da nossa letra, fazer o processamento e, a partir disso, estimar a letra correspondente.



A partir do gabarito que você tem, a IA conseguirá confirmar ou não o resultado alcançado.

Sempre que a rede neural erra, ela ajusta os cálculos aplicados internamente para se adaptar e tentar acertar. Com a repetição desse processo, a taxa de acerto vai aumentando.

Passo 5 – Preencher o CAPTCHA

Com todo o processo feito, basta automatizarmos o processo de preenchimento do CAPTCHA.

O objetivo dessa aula foi de te explicar o processo e a lógica para construção do projeto de quebra de CAPTCHA. Para acompanhar o desenvolvimento do projeto, acesse a nossa [playlist clicando aqui](#).



PYTHON

Módulo 55

INTEGRAÇÃO PYTHON COM ARCGIS



Antes de tudo... o que é o ArcGIS? Essa é uma ferramenta que cada vez mais vem sendo usada no mercado para localização, mapas, rotas e mais uma série de informações. Se não conhece e deseja conhecer, é só [clique aqui](#).

Algumas ressalvas importantes sobre o ArcGIS:

- Não é um software 100% gratuito;
- Até o momento da elaboração dessa apostila, o sistema não suporta versão do Python superior a 3.6;
- Vamos precisar de um ambiente virtual para utilização do Python.

Assim como aprendemos no módulo anterior, vamos começar criando um ambiente virtual dentro de uma pasta **Python e ArcGIS**.

```
(arcgispython) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Python e ArcGis>python -V
Python 3.6.13 :: Anaconda, Inc.

(arcgispython) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Python e ArcGis>pip install jupyter notebook
```

Vamos agora instalar dentro desse mesmo ambiente virtual a Instalação da API do ArcGIS.

Usando as informações fornecidas pela documentação da API, vamos usar o código abaixo para realizar a instalação (demora um pouquinho 😊):

Conda install – c esri arcgis

```
Anaconda Prompt (anaconda3) - conda deactivate - conda deactivate novoambiente - conda deactivate novoambiente - conda deactivate - conda install -c esri arcgis  
(arcgispython) C:\Users\danie\OneDrive\Área de Trabalho\Cartilhas Python\Python e ArcGis>conda install -c esri arcgis
```

Feita a instalação do ArcGis, vamos abrir nosso Jupyter e acessar a nossa pasta **Python e ArcGis**.

ATENÇÃO: Abra via prompt do Anaconda dentro do nosso ambiente virtual criado.



Vamos agora para a Integração em si.

Com Jupyter aberto DENTRO do seu ambiente virtual, vamos começar a comunicar nosso código com a API do ArcGIS.

Essencialmente o que faremos aqui é uma parte do que pode ser aprendido por meio das documentações do próprio ArcGis indicadas abaixo:

- <https://developers.arcgis.com/python/>
- <https://developers.arcgis.com/python/guide/install-and-set-up/>

Como sempre, vamos iniciar importando a biblioteca que nos permitirá trabalhar com o ArcGIS. Nesse caso usaremos :

```
from arcgis.gis import GIS
```

Além disso, vamos acessar essa biblioteca com um usuário genérico.

Importando biblioteca que nos permitirá trabalhar com ArcGGIS

```
from arcgis.gis import GIS
```

```
gis = GIS()
```

Ao não fornecermos nenhum parâmetro, usaremos um usuário genérico

```
user = gis.users.get('rxsingh')  
display(user)
```

Consultando o usuário genérico que estamos usando



Rohit Singh

Bio:

First Name: Rohit

Last Name: Singh

Username: rxsingh

Joined: July 02, 2014

Um baita de um bigode

Agora “logados” como nosso usuário bigodudo vamos para um código um pouco mais útil.

Usando a biblioteca do ArcGIS, podemos por exemplo exibir um mapa interativo.

Utilizando o método `.add_layer()` conseguimos acrescentar por exemplo marcações de trilhas existentes da região.

Lembrando que não estamos focando em como usar o ArcGIS e sim na integração do Python com essa ferramenta.

Em resumo, a premissa será a mesma das demais integrações que utilizamos:

- 1) Encontramos a biblioteca que mais nos oferece recursos para a integração;
- 2) Estudamos a biblioteca para conhecer os métodos disponíveis;
- 3) Usamos a biblioteca juntamente com os conhecimentos do Python.

```
map = gis.map("Palm Springs, CA", zoomlevel=12)
display(map)
```



```
map.add_layer(items[1].layers[0])
```



Seguiremos novamente mais uma documentação do ArcGIS, mas agora usaremos um usuário específico ao invés de um usuário genérico.

Para isso, usaremos a estrutura abaixo:

```
gis = GIS('link', 'usuario', 'senha')
```

Novamente vamos buscar trilhas e como podemos ver, ao rodar o código temos a exibição das trilhas marcadas no mapa do ArcGIS.

```
from arcgis.gis import GIS

print("ArcGIS Online Org account")
gis = GIS("https://www.arcgis.com", "arcgis_python", "P@ssword123")
print("Logged in as " + str(gis.properties.user.username))
```

```
ArcGIS Online Org account
Logged in as arcgis_python
```

```
from arcgis.mapping import WebMap
la_parks_trails = WebMap(webmap)
la_parks_trails
```

